gemstone-py Companion Manual

Setup guide, user manual, examples guide, cookbook, codegen, observability, and performance notes

Generated from the Markdown sources in gemstone-py/docs
Assets are local SVG illustrations stored in the repository.

Contents

gemstone-py Documentation

This directory contains the longer-form documentation set for `gemstone-py`. The top-level `README.md` in the repo is still the quickest way to get the package installed, but the files here are meant to answer the questions people ask on day two, day ten, and day forty-five.

Reading Order

If you are new to the project, use this order:

1. Setup Guide
2. User Manual
3. Examples Guide
4. Cookbook
5. Plan3 Feature Map
6. Type-Safe Smalltalk Codegen
7. Framework Adapters
8. Observability
9. Performance
10. Rust Client
11. Funny Introduction

If you want a narrative overview before diving in:

- Medium article — a complete end-to-end guide written in article style

For release preparation:

- Next release draft

If you are already productive and only need answers:

- "Why is my login failing?" → Setup Guide
- "Which transaction policy should I use?" → User Manual
- "Which example should I run first?" → Examples Guide
- "Which plan3 stream maps to which code?" → Plan3 Feature Map
- "How do I do X quickly?" → Cookbook
- "How do I generate typed wrappers from Protocols?" → Type-Safe Smalltalk Codegen
- "How do I reduce round trips without adding an ORM?" → User Manual
- "How do I retry commit conflicts deliberately?" → Cookbook
- "How do I add another web framework?" → Framework Adapters
- "How do I trace or measure GemStone calls?" → Observability
- "Can a Rust application talk to GemStone directly?" → Rust Client

- "How do I inspect an OOP or class?" → User Manual
- "What are the current benchmark numbers?" → Performance
- "I want the whole story, plus jokes." → Funny Introduction
- "Can I run these from VS Code?" → `vscode-gemstone-py-workbench`

What This Docs Set Covers

- installing and configuring `gemstone-py`
- connecting to a GemStone stone from Python
- GemStone-side bootstrap for the shared helper roots
- transaction policies, session scopes, and failure behaviour
- async sessions and FastAPI/Litestar request integration
- typed OOPs, typed `GSCollection` queries, and managed OOP lifetimes
- Protocol-to-Smalltalk code generation with checked-in typed wrappers
- generated wrapper metadata, `.pyi` stubs, async wrappers, and CI drift checks
- bulk selector sends, persistent-root batch reads/writes, and collection batch updates
- explicit transaction retry helpers and user-facing conflict diagnostics
- opt-in scalar value converters for `datetime`, `date`, `Decimal`, `UUID`, and dataclass payloads
- schema fingerprinting for expected roots, GemStone classes, and migration state
- OOP/class inspection and recursive debug dumps
- optional native PyO3 backend installation and backend selection
- a direct Rust client foundation in the separate `gemstone-rs` workspace
- the persistent data helpers:
- `PersistentRoot`
- `GSCollection`
- `GStore`
- `ObjectLog`
- concurrency helpers such as `RCCounter`, `RCHash`, and `RCQueue`
- Flask, Django, FastAPI, and Litestar request-session integration
- tracing, metrics, and slow-operation logs for session calls
- benchmark numbers, release workflows, and the current examples directory
- a plan3 feature map that links major features to modules, examples, and docs
- the companion VS Code workbench for running examples, opening docs, checking backends, launching the Python database explorer, and handing Smalltalk IDE work to Jasper

Current Lightweight Improvements

The recent improvements deliberately keep `gemstone-py` as a thin GemStone client rather than an object mapper. The main additions are explicit helpers that reduce repeated boilerplate or repeated round trips:

- `PersistentRoot.get_many(...)`, `PersistentRoot.update_many(...)`,

`GsDict.get_many(...)`, and `GsDict.update_many(...)` for batch dictionary access.

- `GemStoneSession.bulk_perform_`, *`perform_many_`*, and `PerformCall` for sending one or more selectors across raw OOPs in one evaluated batch.
- `run_transaction_with_retry(...)` and `retrying_transaction(...)` for bounded replay of a whole unit of work after `CommitConflictError`.
- `format_commit_conflict(...)` and structured conflict diagnostics for logs, CLI output, and incident reports.
- `scalar_value_converter_registry()` and `dataclass_to_dict(...)` for explicit value conversion when a GemStone API expects scalar OOP arguments or plain mapping payloads.
- `schema_fingerprint(...)`, `assert_schema_fingerprint(...)`, and `gemstone-migrations fingerprint` for deployment-time checks that a stone has the expected roots, classes, and migration state.

These are intentionally not a hidden identity map. If an application wants a domain model, it can build one on top of these primitives without the client guessing object identity, lifetime, or persistence semantics behind its back.

Visuals

The images under `docs/assets/` are intentionally repository-native SVG files. That gives you a few benefits:

- they render nicely on GitHub
- they can be edited in a normal text diff
- they do not bloat the repository with binary noise
- they can be reused in presentations, blog posts, or generated manuals

Some of the "screenshots" are stylized screenshot illustrations rather than raw captures. That is deliberate: the examples evolve, and the docs should remain easy to maintain.

Print-Friendly Book

The long-form introduction under `funny-introduction/` is structured as a book, with explicit page breaks for a print/export workflow. It is designed to compile to more than one hundred pages when rendered to PDF or another paged format.

Suggested Build / Export Flow

If you want to turn these docs into a PDF bundle later, a practical approach is:

1. keep the Markdown source here as the canonical text
2. render the long introduction as a separate book
3. render the setup guide, manual, examples guide, and cookbook as a smaller companion manual

That split keeps the funny book delightfully excessive and the operational docs pleasantly searchable.

Setup Guide

This guide gets you from "I cloned the repo" to "I can log in to GemStone from Python and run the examples without muttering at the terminal."

What You Need

At minimum:

- Python 3.11 or newer
- access to a GemStone/S 64 stone
- the GemStone client library (`libgcirpc`)
- a username and password that can log into the stone

For the full local development experience, you will also want:

- the repository checkout
- a working virtual environment
- the ability to run live tests against your stone
- if you use the self-hosted workflows, a GitHub Actions runner on the GemStone host

Install the Package

Which Install Path Should I Use?

Use case	Command
Normal users	`python3 -m pip install gemstone-py`

Use case	Command
Native acceleration	`python3 -m pip install "gemstone-py[fast]"`
Django web apps	`python3 -m pip install "gemstone-py[django]"`
Litestar web apps	`python3 -m pip install "gemstone-py[litestar]"`
Source checkout examples/ development	`python3 -m pip install -e ".[examples,dev]"`
VS Code users	`code --install-extension unicompute.gemstone-py-workbench`

Installed Package

From PyPI:

```
python3 -m pip install gemstone-py
```

If you want the optional PyO3 native fast path and a wheel exists for your platform:

```
python3 -m pip install "gemstone-py[fast]"
python -c "from gemstone_py import _gci; print(_gci.IMPLEMENTATION)"
```

From a repository checkout, the fuller backend example is `python -m examples.native_backend.check_backend`.

For web examples from an installed environment:

```
python3 -m pip install "gemstone-py[fastapi]"
gemstone-fastapi-example --reload
```

For Django applications:

```
python3 -m pip install "gemstone-py[django]"
```

For Litestar applications:

```
python3 -m pip install "gemstone-py[litestar]"
gemstone-litestar-example --reload
```

Source Checkout

From a repo checkout:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -U pip
python -m pip install -e .[dev]
```

For source-checkout examples without the full development toolchain:

```
cd /path/to/gemstone-py
source .venv/bin/activate
python -m pip install -e ".[examples]"
python -m examples.fastapi.run --reload
python -m examples.litestar.run --reload
gemstone-examples list
```

Keep the terminal in the checkout root for `python -m examples...` commands. If you want to run examples from another directory, use the installed commands such as `gemstone-examples litestar --reload` or set `PYTHONPATH` to the checkout path explicitly.

Required Environment Variables

The package expects its runtime configuration from environment variables. The minimum set for most local work is:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

`GS_STONE` is the canonical stone variable. `GS_STONE_NAME` is accepted as an alias when `GS_STONE` is absent; setting both to the same value keeps tools that use either spelling in sync.

Common optional variables:

```
export GS_HOST=localhost
export GS_NETLDI=netldi
export GS_GEM_SERVICE=gemnetobject
export GS_HOST_USERNAME=
export GS_HOST_PASSWORD=
export GS_LIB_PATH=/full/path/to/libgcirpc-3.7.x-64.dylib
```

Quick reference:

Variable	Required	Example	Purpose
`GS_LIB`	Yes	`/opt/gemstone/product/lib`	GemStone `lib/` directory for library discovery
`GS_STONE`	Yes	`gs64stone`	Stone name
`GS_STONE_NAME`	No	`gs64stone`	Stone-name alias used by some tools; used when `GS_STONE` is absent
`GS_USERNAME`	Yes	`DataCurator`	GemStone login username
`GS_PASSWORD`	Yes	`swordfish`	GemStone login password
`GS_LIB_PATH`	No	`/full/path/libgcirpc.dylib`	Pin to a specific `libgcirpc` file
`GS_HOST`	No	`localhost`	Remote stone host
`GS_NETLDI`	No	`netldi`	NetLDI service name
`GS_GEM_SERVICE`	No	`gemnetobject`	Gem service name
`GS_HOST_USERNAME`	No		Remote host OS username
`GS_HOST_PASSWORD`	No		Remote host OS password

First Real Login

The simplest sanity check is a tiny Python script:

```
from gemstone_py import GemStoneConfig, GemStoneSession

config = GemStoneConfig.from_env()

with GemStoneSession(config=config) as session:
    print(session.eval("1 + 2"))
```

If that prints `3`, the package can:

- read your configuration
- load `libgcirpc`
- log in to GemStone

- evaluate Smalltalk in the repository

That is enough to start.

Recommended First Commands

The installed package ships with a few small CLI helpers:

```
gemstone-hello
gemstone-smalltalk-demo
gemstone-examples hello
gemstone-examples smalltalk-demo
gemstone-examples value-converters
gemstone-bootstrap --status
gemstone-benchmarks --help
```

From a repository checkout, these examples are also useful first checks:

```
python -m examples.native_backend.check_backend
```

What they are good for:

- `gemstone-hello`

  Tiny smoke check for the installed package.

- `gemstone-smalltalk-demo`

  Basic bridge demo that is easy to reason about.

- `gemstone-examples ...`

  A stable wrapper for the example entry points.

- `gemstone-examples value-converters`

  Offline preview of opt-in scalar converters and dataclass payload conversion.

- `gemstone-bootstrap --status`

  Checks whether the GemStone-side helper roots are already present.

- `gemstone-benchmarks`

  The maintained benchmark lane, distinct from the teaching examples.

- `examples.native_backend.check_backend`

  Shows whether the current process selected ctypes or the optional native GCI backend.

GemStone-Side Bootstrap

Most helpers create their own GemStone-side roots lazily, but an explicit bootstrap step is useful for new stones, shared demo stones, and onboarding checks. The packaged command ships a small Smalltalk file and evaluates it once against the configured stone:

```
gemstone-bootstrap --status
gemstone-bootstrap --dry-run
gemstone-bootstrap
```

The command creates missing helper roots and writes a bootstrap version marker:

Key	Class	Used by
`GemstonePyBootstrapVersion`	`String`	Bootstrap/version audit
`GStoreRoot`	`StringKeyValueDictionary`	`gemstone_py.gstore.GStore`
`GSQueryRoot`	`Dictionary`	`gemstone_py.gsquery.GSCollection`

It does not replace existing helper roots. If your stone already has `GStoreRoot` or `GSQueryRoot`, the command reports them as present and leaves them untouched. `ObjectLogEntry objectLog` is checked as a GemStone built-in object log, not created by the bootstrap script.

Transaction Policy: Read This Early

The most important behavioural rule in `gemstone-py` is that transaction intent should be explicit.

`GemStoneSession(...)` defaults to manual transaction control.

That means:

- a plain session does not silently commit just because your `with` block ended
- write scripts must call `commit()` explicitly or use a scoped helper that commits on success
- read-only scripts should usually abort or use an abort-on-exit policy

The common options are:

```
from gemstone_py import GemStoneSession, TransactionPolicy

with GemStoneSession(
    config=config,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
```

```
) as session:
    ...
```

Or:

```
from gemstone_py import session_scope

with session_scope(config=config) as session:
    ...
```

Use this rule of thumb:

- one-off write script -> `COMMIT_ON_SUCCESS`
- read-only inspection -> `ABORT_ON_EXIT`
- library or framework code -> manual, unless you deliberately wrap it

First Useful Persistent Write

Once login works, prove that you can store and read data:

```
from gemstone_py import GemStoneConfig, TransactionPolicy
from gemstone_py import GemStoneSession
from gemstone_py.persistent_root import PersistentRoot

config = GemStoneConfig.from_env()

with GemStoneSession(
    config=config,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
) as session:
    root = PersistentRoot(session)
    root["DocsSmokeTest"] = {"status": "ok", "kind": "setup-guide"}

with GemStoneSession(config=config) as session:
    root = PersistentRoot(session)
    print(root["DocsSmokeTest"])
```

If that round trip works, the rest of the package will feel much less mysterious.

First Async Check

If your application is async-first, run the async example after the basic sync login works:

```
python -m examples.async_features.session_root_and_collection
```

The example opens an `AsyncSession`, writes through `AsyncPersistentRoot`, keeps a managed OOP alive, and exercises `AsyncGSCollection`.

For FastAPI:

```
python -m pip install -e ".[examples]"
python -m examples.fastapi.run --reload
```

When the server starts, you should see output like:

```
INFO:     Will watch for changes in these directories: ['/path/to/gemstone-py']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [49045] using WatchFiles
INFO:     Started server process [49048]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

With that server running, test it from a second terminal.

Basic checks:

```
curl -i http://127.0.0.1:8000/
```

Expected:

```
HTTP/1.1 200 OK
```

Body should include:

```
{"name":"gemstone-py FastAPI example","endpoints":{"health":"/health/
gemstone","docs":"/docs","openapi":"/openapi.json"}}
```

Then test the GemStone endpoint:

```
curl -i http://127.0.0.1:8000/health/gemstone
```

Expected if GemStone credentials/environment are set and the stone is reachable:

```
{"result":7}
```

Also open these in a browser:

```
http://127.0.0.1:8000/
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/health/gemstone
```

Typed and Lifetime Examples

Once basic reads and writes work, these examples show the newer type and export-set APIs:

```
python -m examples.typed_access.typed_oops_and_queries
python -m examples.lifetime.managed_oop_handles
```

Use them to confirm that `TypedOop[T]`, typed `GSCollection` predicates, `execute_managed(...)`, and `session.handle(...)` behave against your stone.

Running the Tests

Unit tests:

```
python -m unittest discover -s tests -p 'test*.py'
```

The standard local verification lane:

```
./scripts/run_ci_checks.sh
```

Generated wrapper drift check:

```
./scripts/check_codegen.sh
```

Live GemStone tests:

```
GS_RUN_LIVE=1 ./scripts/run_live_checks.sh
```

The live script prints the module coverage list before running. That makes it easier to notice when a new live integration module was added but not wired into the maintained lane.

Rust-core native smoke for `gemstone-py-native`:

```
python scripts/run_native_rust_core_live_smoke.py --dry-run
GS_RUN_LIVE=1 python scripts/run_native_rust_core_live_smoke.py --require-live
```

The dry run verifies the installed PyO3 extension exposes the `RustCoreSession` bridge and `rust_core_*_json()` reports. The live run logs in through `gemstone_rs::py_native`, checks `3 + 4`, performs `printString`, round-trips a string, writes and aborts a temporary global, and logs out.

Longer live soak tests:

```
GS_RUN_LIVE=1 GS_RUN_LIVE_SOAK=1 ./scripts/run_live_checks.sh
```

Destructive live tests are intentionally separated. They mutate shared state and are not meant to be run casually.

Troubleshooting

`GemStoneConfigurationError`

Usually means `GS_USERNAME` and/or `GS_PASSWORD` are missing.

`OSError` while loading the client library

Usually one of:

- `GS_LIB` is wrong
- `GS_LIB_PATH` points at a missing or incompatible `libgcirpc`
- the local machine does not actually have the client library installed

Login works in one shell but not another

Check that both shells export the same environment values. This failure mode is common, boring, and surprisingly effective.

Your writes "worked" but the data is missing

That almost always means you forgot to commit or assumed a session would commit for you. Re-check the transaction policy.

A Flask request wrote partial data after a handled error

Use the request-session integration from `gemstone_py.web` and let request teardown own the final commit-or-abort decision. The web helpers were hardened exactly to avoid this problem.

The native backend is not selected

Check the current process:

```
python -m examples.native_backend.check_backend
```

If `selected backend` is `ctypes`, either the optional package is not installed or `GEMSTONE_PY_GCI_BACKEND=ctypes` was set before Python started. Install `gemstone-py[fast]` and start a fresh Python process. Use `GEMSTONE_PY_GCI_BACKEND=native` when you want import failure instead of silent fallback.

Where to Go Next

- User Manual for the main abstractions
- Examples Guide for the runnable demos
- Cookbook for copy-paste-friendly recipes
- Codegen for generated typed wrappers

User Manual

This manual explains the public surface of `gemstone-py` from the point of view of a user who wants to build reliable code, not just make a demo print a nice dictionary once.

The Mental Model

Three ideas matter more than the rest:

1. Python code drives the application.
2. GemStone stores the persistent state.
3. Transactions are explicit enough that you can explain them to another human.

`gemstone-py` is not trying to hide GemStone completely. It gives you a Pythonic surface, but it still wants you to understand:

- when sessions are opened and closed
- when transactions commit or abort
- which data structures are persisted in GemStone
- what happens when two sessions try to change the same thing

That restraint is a strength. The library does not pretend that distributed state is a teddy bear.

Core Building Blocks

GemStoneConfig

`GemStoneConfig` is the runtime configuration object. In most applications you will construct it from the environment:

```
from gemstone_py import GemStoneConfig

config = GemStoneConfig.from_env()
```

Use it when:

- you want a single source of connection settings
- you need to pass the same config into sessions, stores, or web providers
- you want to validate missing credentials early

GemStoneSession

This is the direct session object. Use it when you want explicit session and transaction control.

Typical pattern:

```
from gemstone_py import GemStoneSession, TransactionPolicy

with GemStoneSession(
    config=config,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
) as session:
    print(session.eval("System myUserProfile userId"))
```

Useful methods include:

- `eval(...)`

  Evaluate Smalltalk code directly.

- `commit()`

  Commit the current transaction.

- `abort()`

  Abort the current transaction.

- `global_get(...)`

  Resolve named objects from the repository.

- `bulk_perform_value(...)` / `bulk_perform_oop(...)`

  Send one selector to many raw OOP receivers in one evaluated batch.

- `bulk_perform_calls_value(...)` / `bulk_perform_calls_oop(...)`

  Send mixed selector calls in one batch when each receiver or selector differs.

When to use it:

- scripts
- lower-level tooling
- integration code that must control failure behaviour precisely

Bulk Selector Sends

Bulk sends are for the cases where you already have raw OOPs and would otherwise call `perform_*` repeatedly in a loop. They keep the API explicit: you still choose selectors and raw OOP arguments, but the package evaluates the batch in one GemStone round trip.

```
from gemstone_py import PerformCall

with GemStoneSession(config=config) as session:
    order_oops = [session.global_get("OrderA"), session.global_get("OrderB")]
    statuses = session.bulk_perform_value(order_oops, "status")

    labels = session.bulk_perform_calls_value([
        PerformCall(order_oops[0], "printString"),
        (order_oops[1], "status"),
    ])
```

Use `perform_many_value(...)` and `perform_many_oop(...)` when that reads more naturally in application code. They are aliases for the bulk selector helpers. `AsyncSession` exposes the same bulk and perform-many names as `async` methods.

`session_scope(...)`

This is the higher-level unit-of-work helper. It is usually the right answer for application code.

```
from gemstone_py import session_scope

with session_scope(config=config) as session:
    ...
```

Why it exists:

- it makes commit-on-success intent obvious
- it composes naturally with service-layer code

- it avoids sprinkling `commit()` everywhere like nervous confetti

AsyncSession

`gemstone_py.aio.AsyncSession` is the async facade for applications built on `asyncio`, FastAPI, or Starlette-style request handling.

It does not make GCI itself async. Instead, it runs all calls for one session on one dedicated worker thread. That keeps the GemStone session on its owning thread while letting your event loop continue serving other work.

```
from gemstone_py import GemStoneConfig
from gemstone_py.aio import AsyncSession, AsyncSessionPool

config = GemStoneConfig.from_env()

async with AsyncSession.connect(config=config) as session:
    value = await session.eval("3 + 4")

    async with session.transaction():
        root = session.root()
        await root.set("AsyncManualExample", {"value": value})
```

Use `AsyncSession` when:

- request handlers are already async
- you want FastAPI dependency injection
- you need to keep blocking GCI work out of the event loop

Use `AsyncSessionPool` when async requests should reuse logged-in GemStone sessions instead of opening one session per request:

```
pool = AsyncSessionPool(
    maxsize=8,
    minsize=2,
    config=config,
    idle_timeout_seconds=900,
    validation_query="1 + 1",
    validation_interval_seconds=60,
)

async with pool.acquire() as session:
    value = await session.eval("3 + 4")

await pool.close()
```

The runnable repository example is `examples/async_features/session_root_and_collection.py`.

Transaction Policies

`TransactionPolicy` exists so you do not have to guess what a context manager will do.

The main values are:

- `MANUAL`
- `COMMIT_ON_SUCCESS`
- `ABORT_ON_EXIT`

Recommended use:

Situation	Policy
low-level explicit control	`MANUAL`
script that writes data	`COMMIT_ON_SUCCESS`
read-only inspection	`ABORT_ON_EXIT`
request/response web unit of work	`COMMIT_ON_SUCCESS` via `session_scope(...)` or request integration

`PersistentRoot`

`PersistentRoot` is the friendliest entry point into repository-backed state.

The default instance is the user's `UserGlobals` dictionary:

```
from gemstone_py.persistent_root import PersistentRoot

root = PersistentRoot(session)
root["CustomerCount"] = 12
```

Other dictionaries are also available:

- `PersistentRoot.globals(session)`
- `PersistentRoot.published(session)`
- `PersistentRoot.session_methods(session)`

Use `PersistentRoot` when:

- you want a stable named entry point
- you are storing dictionaries, counters, queues, or domain objects
- you want something simple and direct

Use something more specialized when:

- you need indexed search across many rows -> `GSCollection`
- you want a file-like key/value store abstraction -> `GStore`
- you want append-only event style logging -> `ObjectLog`

Good Practices With `PersistentRoot`

- keep the top-level key space tidy
- use descriptive names
- group related data under one top-level bucket when it makes sense
- do not turn `UserGlobals` into a junk drawer with no naming discipline

Batch Reads And Writes

Use `get_many(...)` and `update_many(...)` when a request, migration, or maintenance script needs several keys at once. The helpers avoid a Python loop of individual repository sends while keeping the operation visible in code.

```
with session_scope(config=config) as session:
    root = PersistentRoot(session)

    values = root.get_many(["CustomerCount", "Settings"], default=None)
    root.update_many({
        "CustomerCount": int(values["CustomerCount"] or 0) + 1,
        "LastSweepAt": "2026-05-14T22:00:00Z",
    })

    if "Settings" not in root:
        root["Settings"] = {}
    settings = root["Settings"]
    settings.update_many({"theme": "dark", "page_size": 50})
```

`PersistentRoot` uses symbol keys for top-level `UserGlobals` access. Nested `GsDict` values use string keys. That distinction mirrors the GemStone objects being wrapped rather than hiding them behind object mapping. `AsyncPersistentRoot` exposes async `get_many(...)` and `update_many(...)` counterparts for async applications.

For persistent ordered lists, `OrderedCollection.extend(...)` adds several values with one `addAll:` send:

```
from gemstone_py import session_scope
from gemstone_py.ordered_collection import OrderedCollection

with session_scope(config=config) as session:
    collection = OrderedCollection(session)
    collection.extend(["draft", "review", "paid"])
```

```
    root = PersistentRoot(session)
    root["InvoiceStates"] = collection
```

GSCollection

`GSCollection` is the indexed collection helper. It is the right tool when you need more than "look up a dictionary by a single well-known key."

Use it for:

- indexed search
- filtering by stored attributes
- application collections with repeatable lookup patterns

Typical pattern:

```
from gemstone_py.gsquery import GSCollection

people = GSCollection("People", config=config)
people.insert({"@name": "Tariq", "@city": "London"})
people.add_index("@city")
matches = people.search("@city", "eql", "London")
```

Think of it as the package's middle ground between:

- raw repository objects
- and a full ORM that wants to restructure your life

Typed Queries

`GSCollection.query(...)` can accept a Python `Protocol` as a type witness. The query still runs against GemStone indexed paths, but your Python code gets attribute names and better editor help.

```
from typing import Protocol
from gemstone_py.gsquery import GSCollection

class BlogPostRecord(Protocol):
    title: str
    status: str

posts = GSCollection("SimplePosts", config=config).query(BlogPostRecord)
published = posts.where(lambda post: post.status == "published").all()

for post in published:
    print(post.title)
```

For large result sets, use `iter(chunk_size=...)` to fetch records in bounded batches. `all()` keeps the familiar list return value, but it materializes that list by consuming the same chunked iterator path:

```
for post in posts.where(lambda post: post.status ==
"published").iter(chunk_size=500):
    print(post.title)
```

The lambda is a query builder expression, not a live object callback. `post.status` becomes the GemStone ivar path `@status`.

See `examples/typed_access/typed_oops_and_queries.py` for a runnable typed query and `examples/typed_access/simple_blog_queries.py` for importable helper functions.

Typed OOPs and Lifetime Handles

Raw integer OOPs remain supported for compatibility. New code can opt into typed or managed wrappers when the extra structure is useful.

`TypedOop[T]` carries a phantom Python type:

```
from typing import Protocol
from gemstone_py import GemStoneSession, gemstone_class

@gemstone_class("Date")
class GemStoneDate(Protocol):
    @property
    def printString(self) -> str: ...

with GemStoneSession(config=config) as session:
    today = session.execute_typed("Date today", GemStoneDate)
    print(today.proxy().printString)
```

Generated Typed Wrappers

When a Protocol describes method-shaped GemStone access, use `gemstone-codegen` to generate a concrete wrapper instead of repeating Smalltalk strings throughout the application.

```
from typing import Protocol
from gemstone_py import gemstone_class, gemstone_selector

@gemstone_class("OkzBooking", async_=True)
class OkzBookingProto(Protocol):
    status: str

    @classmethod
    @gemstone_selector("findById:")
```

```
    def find_by_id(cls, booking_id: str) -> "OkzBookingProto":
        ...

    def mark_paid(self, at_posix_seconds: int) -> None:
        ...
```

Generate and check in the wrapper package:

```
gemstone-codegen \
  --module examples.typed_access.codegen_demo.models \
  --output examples/typed_access/codegen_demo/generated \
  --clean
```

Application code can then use regular Python methods:

```
from examples.typed_access.codegen_demo.generated import OkzBooking

with GemStoneSession(config=config) as session:
    booking = OkzBooking.find_by_id(session, "B-1001")
    print(booking.status)
    booking.mark_paid(1_779_912_000)
```

See `docs/codegen.md` for selector rules, async wrappers, CI `--check` usage, and the VS Code Codegen Explorer flow that consumes the database explorer's `/codegen/*` endpoints for live discovery, source lookup, selection JSON, and preview generation. Generated classes also expose `__gemstone_protocol__` and `__gemstone_selectors__` so diagnostics, admin UI, and code review tooling can identify where a wrapper came from without parsing generated source files.

For object lifetime, use `execute_managed(...)` when a raw OOP should stay in GemStone's export set while Python holds a handle:

```
with GemStoneSession(config=config) as session:
    collection = session.execute_managed("OrderedCollection new")
    collection.send("add:", session.new_string("kept alive"))
    print(collection.print_string())
    collection.close()
```

Use `session.handle(oop)` when you already have a raw OOP and want an explicit scope:

```
with session.handle(raw_oop) as handle:
    print(handle.send("printString"))
```

`execute()` and `perform()` still return raw OOP integers. The managed variants are additive and make lifetime intent visible.

The runnable repository example is `examples/lifetime/managed_oop_handles.py`.

Explicit Value Converters

`gemstone-py` does not globally convert every Python object it sees. When a GemStone API expects scalar-ish OOP arguments, opt into a small converter registry at the call site:

```
from datetime import date
from decimal import Decimal
from gemstone_py import scalar_value_converter_registry

registry = scalar_value_converter_registry()

with GemStoneSession(config=config) as session:
    invoice_oop = session.global_get("CurrentInvoice")
    due_on, amount = registry.to_oops(session, [
        date(2026, 5, 15),
        Decimal("19.95"),
    ])
    session.perform_oop(invoice_oop, "dueOn:amount:", due_on, amount)
```

The built-in registry covers `datetime`, exact `date`, `Decimal`, and `UUID`. Use `registry.copy()` and `registry.extend(...)` when an application needs its own explicit adapters.

For dataclasses, convert to a plain payload before persisting or sending it:

```
from gemstone_py import dataclass_to_dict

payload = dataclass_to_dict(booking_patch, recurse=False)
root["PendingBookingPatch"] = payload
```

This is deliberately value conversion, not object mapping. It does not create an identity map, track dirty fields, or decide when domain objects should be written.

`GStore`

`GStore` is a GemStone-backed key/value store with its own transactional behaviour around store operations.

Use it when:

- you want a store-like API
- you want separate named stores
- you want a compact way to persist structured payloads

Typical pattern:

```
from gemstone_py.gstore import GStore

store = GStore("inventory.db", config=config)
store["sku:123"] = {"name": "Hat", "stock": 8}
print(store["sku:123"])
```

`GStore` is especially handy in examples, utilities, and small application components that want a store-shaped abstraction without building a full domain model first.

`ObjectLog`

`ObjectLog` is the package's event-log helper. It is good for:

- audit trails
- structured append-only logging
- "what happened?" questions after a workflow ran

Typical pattern:

```
from gemstone_py.objectlog import ObjectLog

log = ObjectLog(config=config)
log.info("user_created", {"user": "tariq", "source": "manual"})
for entry in log.entries():
    print(entry)
```

This is not a replacement for Python logging. It is a repository-resident event log for when the record should live with the data.

Concurrency Helpers

The concurrency helpers give you shared, repository-backed coordination primitives:

- `RCCounter`
- `RCHash`
- `RCQueue`

These are useful when you need shared mutable state between sessions and want the concurrency semantics to live in GemStone instead of in a Python process.

Use them carefully:

- they are powerful
- they make contention visible
- they will teach you honesty about multi-session behaviour

The package also includes:

- nested transaction helpers
- conflict detection helpers
- instance listing utilities

Inspect And Debug

Use inspection helpers when you have a raw OOP or class name and need to see what GemStone is actually holding:

```
with GemStoneSession(config=config) as session:
    booking = session.execute("OkzBooking findById: 'B-1001'")

    inspected = session.inspect(booking)
    print(inspected.class_name, inspected.summary)

    dump = session.dump(booking, depth=2)
    print(dump["slots"])

    description = session.describe_class("OkzBooking")
    print(description.instvars)
```

The command-line form uses the same implementation:

```
gemstone-inspect --oop 123456789
gemstone-inspect --oop 123456789 --dump --depth 2
gemstone-inspect --class OkzBooking --json
```

`inspect()` returns a one-level `InspectionResult`; slot values are `InspectedReference` objects with OOP, class name, and summary. `dump()` follows those references recursively and marks cycles. Use `slots=...`, `classes=...`, and `max_slots=...` when the object graph is large or when a CLI report should stay readable:

```
dump = session.dump(
    booking,
    depth=3,
    slots=["status", "customer"],
    classes=["OkzBooking", "OkzCustomer"],
    max_slots=20,
)
```

The equivalent CLI flags are `--slot`, `--include-class`, `--max-slots`, and `--json`.

Migrations

Use module-style migrations when repository objects or GemStone-side classes need to evolve in a controlled order. A manifest lists migration modules, and each module exposes `upgrade(session)` plus optional `downgrade(session)`.

```
gemstone-migrations scaffold add_amount_to_booking --directory migrations
gemstone-migrations status --manifest my_app.migrations.manifest
gemstone-migrations upgrade --manifest my_app.migrations.manifest --dry-run
gemstone-migrations upgrade --manifest my_app.migrations.manifest --dry-run --record
gemstone-migrations upgrade --manifest my_app.migrations.manifest
```

Plain `--dry-run` reports the pending migration ids. `--dry-run --record` also runs migration callbacks against a `RecordingMigrationSession`, prints common session calls, and never sends those calls to GemStone:

```
dry-run upgrade: 1 step(s)
  002_add_amount_to_booking
recorded operations:
  # upgrade 002_add_amount_to_booking
  session.eval("OkzBooking addInstVarName: 'amount' ifAbsent: [ nil ]")
  session.commit()
```

The recorder is deliberately conservative. It returns placeholder OOPs and is best for migrations made of direct `session.eval(...)`, `execute(...)`, `perform_value(...)`, `commit()`, and `abort()` calls. Use a staging stone for migrations that branch on live data.

Applied versions are tracked under `GemstonePyMigrations` in `UserGlobals`. Before applying new steps, the runner checks that the live version table still matches the local manifest and stored migration checksums.

Real `upgrade` and `downgrade` runs acquire an advisory lock in `UserGlobals` before applying steps. Dry-runs do not lock. If a previous process crashed and left a stale lock, use `--force-lock` with a clear owner string:

```
gemstone-migrations upgrade --manifest my_app.migrations.manifest \
  --force-lock --lock-owner release-2026-05-11
```

For class-shape changes, compare a live GemStone class with a local Protocol or type witness:

```
gemstone-migrations diff-class OkzBooking \
  --local-class my_app.models:OkzBookingProto
```

The output is advisory: review the suggested instance-variable changes before copying them into a migration.

For deployment checks, fingerprint the expected repository shape without mutating the stone:

```
gemstone-migrations fingerprint \
  --root Bookings \
  --root BookingIndexes \
  --class OkzBooking \
  --manifest my_app.migrations.manifest \
  --json
```

Application startup or release automation can use the Python API when it needs to compare against a known digest:

```
from gemstone_py.migrations import assert_schema_fingerprint, load_manifest

steps = load_manifest("my_app.migrations.manifest")
assert_schema_fingerprint(
    session,
    expected_digest="...",
    root_keys=["Bookings", "BookingIndexes"],
    class_names=["OkzBooking"],
    migration_steps=steps,
)
```

The live test lane includes a module migration round trip:

```
GS_RUN_LIVE=1 ./scripts/run_live_checks.sh
```

Commit Conflicts

When two sessions modify the same object and both try to commit, GemStone raises a conflict. The second committer gets a `CommitConflictError`.

This is not a bug. It is the correct behaviour of an optimistic concurrency system. The right response is:

1. catch `CommitConflictError`
2. call `session.abort()` to reset the transaction
3. reload the data
4. reapply the change (with a narrowed scope if possible)
5. retry the commit

Keep retries bounded. Log them. If you see frequent conflicts on the same object, that is a signal the write pattern needs rethinking — not that the concurrency model is wrong.

```
from gemstone_py import GemStoneConfig, PersistentRoot, retrying_transaction

config = GemStoneConfig.from_env()
```

```
def increment_visits(session):
    root = PersistentRoot(session)
    root["Visits"] = int(root.get("Visits", 0)) + 1
    return root["Visits"]

new_value = retrying_transaction(increment_visits, config=config, attempts=5)
```

The retry helper takes a callable rather than a context manager because a real retry must replay the whole unit of work after aborting and reloading state. Use `on_conflict=` when you want structured logging:

```
def log_conflict(retry):
    print(retry.format(include_summaries=False))

retrying_transaction(
    increment_visits,
    config=config,
    attempts=5,
    on_conflict=log_conflict,
)
```

For lower-level handlers, `CommitConflictError` exposes `diagnostics(session)` and `format(session)`. The top-level `format_commit_conflict(...)` helper renders the same report when you already have the exception object.

Web Integration

The web integration lives in `gemstone_py.web`.

The main pieces are:

- `install_flask_request_session(...)`
- `GemStoneSessionPool`
- `GemStoneThreadLocalSessionProvider`
- `session_scope(...)`
- `gemstone_py.session_providers`
- `gemstone_py.frameworks.django`
- `gemstone_py.frameworks.flask`
- `gemstone_py.web_core.RequestScope`
- `gemstone_py.web_core.AsyncRequestScope`

The Flask and FastAPI adapters now share `gemstone_py.web_core`, a framework-neutral lifecycle layer. `RequestScope` and `AsyncRequestScope` own the lazy session acquisition, commit-or-abort decision, and provider release or logout path. Use those primitives when adding another

framework adapter instead of copying Flask teardown code. The reusable sync providers live in `gemstone_py.session_providers`; the older `gemstone_py.web` and top-level imports remain compatibility re-exports.

One Session Per Request

If you are building a Flask app, use the request integration so the request lifecycle decides the final transaction outcome.

That avoids the worst class of web persistence bugs:

- request handled an exception
- response still rendered
- persistence layer silently committed partial work anyway

The package already corrected this behaviour at the framework layer, so use it.

Pool vs Thread-Local

Use `GemStoneSessionPool` when:

- you have multiple request workers
- you want bounded reuse
- you want production-friendly pooling semantics
- stale sessions should be checked with a cheap validation query before reuse
- idle sessions should be evicted instead of sitting in the pool indefinitely

```
provider = GemStoneSessionPool(
    maxsize=4,
    minsize=1,
    config=GemStoneConfig.from_env(),
    idle_timeout_seconds=900,
    idle_sweep_interval_seconds=60,
    validation_query="1 + 1",
    validation_interval_seconds=60,
)

stats = provider.stats()
print(stats.in_use, stats.idle, stats.created_total, stats.evicted_total)
```

`minsize` is the idle-retention floor for maintenance: sweeps will not evict below that many live sessions. Use `warm(count)` or Flask's `warmup_sessions` when you want eager login at application startup.

Use `GemStoneThreadLocalSessionProvider` when:

- your threading model is simple and stable
- one session per thread is the clearest fit

FastAPI

FastAPI integration lives in `gemstone_py.aio.fastapi`.

```
from fastapi import Depends, FastAPI
from gemstone_py import GemStoneConfig
from gemstone_py.aio import AsyncSession, AsyncSessionPool
from gemstone_py.aio.fastapi import pool_session_dependency, session_dependency

app = FastAPI()
get_gemstone = session_dependency(config=GemStoneConfig.from_env())

@app.get("/health/gemstone")
async def gemstone_health(session: AsyncSession = Depends(get_gemstone)):
    return {"result": await session.eval("3 + 4")}
```

For busier FastAPI apps, use a pool-backed dependency:

```
pool = AsyncSessionPool(maxsize=8, minsize=2, config=GemStoneConfig.from_env())
get_pooled_gemstone = pool_session_dependency(pool)
```

Use this when your web stack is async-first. Use the Flask integration when your application is Flask-first.

The runnable repository example is `examples/fastapi/app.py`.

Django

Django integration lives in `gemstone_py.frameworks.django`. The module does not import Django directly; it exposes middleware and a `request_session(...)` helper that work with Django's standard request object.

```
from django.http import JsonResponse
from gemstone_py import GemStoneConfig
from gemstone_py.frameworks.django import GemStoneSessionMiddleware, request_session

def gemstone_session_middleware(get_response):
    return GemStoneSessionMiddleware(get_response, config=GemStoneConfig.from_env())

def health(request):
    session = request_session(request)
    return JsonResponse({"result": session.eval("3 + 4")})
```

Install the optional framework dependency with:

```
python -m pip install "gemstone-py[django]"
```

Litestar

Litestar integration lives in `gemstone_py.aio.litestar`. The module does not import Litestar directly; it provides async dependency callables and ASGI middleware that application code can wire into Litestar.

```
from litestar import Litestar, get
from litestar.di import Provide
from gemstone_py import GemStoneConfig
from gemstone_py.aio.litestar import pool_session_dependency, session_dependency

get_gemstone = session_dependency(config=GemStoneConfig.from_env())
get_pooled_gemstone = pool_session_dependency(pool)

@get("/health/gemstone", dependencies={"session": Provide(get_gemstone)})
async def gemstone_health(session):
    return {"result": await session.eval("3 + 4")}

app = Litestar(route_handlers=[gemstone_health])
```

Install the optional framework dependency with:

```
python -m pip install "gemstone-py[litestar]"
```

The runnable repository example is `examples/litestar/app.py`. Use `gemstone-examples list` or `examples/cookbook/README.md` when choosing between FastAPI, Litestar, Flask, Django, and lower-level examples.

Adapter Core

For a custom sync framework, keep one `RequestScope` in request-local state and finalize it during teardown:

```
from gemstone_py import GemStoneConfig, RequestScope, TransactionPolicy
from gemstone_py.session_providers import GemStoneSessionPool

pool = GemStoneSessionPool(maxsize=4, config=GemStoneConfig.from_env())

scope = RequestScope(
    session_provider=pool,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
)
session = scope.session()
try:
    session.eval("3 + 4")
finally:
    scope.finalize()
```

For an async framework, use `AsyncRequestScope` with `AsyncSessionPool` and await `scope.finalize(...)` from the framework's request cleanup hook.

See `docs/framework-adapters.md` for a concise sync and async adapter recipe.

Benchmarks and Build Lanes

This package has both examples and maintained operational lanes. Keep them separate in your head.

Examples are for:

- learning
- translation exercises
- inspection

The benchmark lane is for:

- reproducible measurement
- stored JSON reports
- baseline comparison
- GitHub workflow enforcement

The CLI is:

```
gemstone-benchmarks --entries 500 --search-runs 20
```

Release and Packaging Surface

The public install surface is the `gemstone_py` package.

The package now has:

- PyPI publishing
- TestPyPI rehearsal
- post-release verification against real PyPI
- installed-artifact API contract checks
- local PyPI/TestPyPI index verification with `gemstone-publish-verify`
- optional PyO3 native wheels through `gemstone-py[fast]`

That means you can treat the package as a real distributable unit, not just a working directory with ambition.

After publishing, verify both public indexes from a clean shell:

```
gemstone-publish-verify --gemstone-version 0.2.12 --native-version 0.1.2
```

The command checks project JSON, version-specific JSON, the simple index, and temporary-virtualenv installs for PyPI and TestPyPI.

Native Fast Path

The default install uses the pure-ctypes GCI backend:

```
python -m pip install gemstone-py
```

The optional native fast path installs the PyO3 extension package:

```
python -m pip install "gemstone-py[fast]"
```

Backend selection happens at import time. For comparisons, set `GEMSTONE_PY_GCI_BACKEND=ctypes` or `GEMSTONE_PY_GCI_BACKEND=native` before starting Python.

The runnable repository example is `examples/native_backend/check_backend.py`.

For the Rust-backed shared-core bridge inside `gemstone-py-native`, run:

```
python scripts/run_native_rust_core_live_smoke.py --dry-run
GS_RUN_LIVE=1 python scripts/run_native_rust_core_live_smoke.py --require-live
```

Use the dry run in wheel/sdist packaging checks. Use the live command before publishing native wheels that are expected to exercise `gemstone_rs::py_native`.

Recommended Adoption Path

If you are bringing `gemstone-py` into a project, a sensible order is:

1. start with `GemStoneConfig` + `GemStoneSession`
2. move most application work into `session_scope(...)`
3. use `PersistentRoot` first for obvious top-level state
4. introduce `GSCollection` only when indexed queries are justified
5. add `TypedOop[T]`, typed queries, or managed handles when the code benefits

   from clearer object identity and lifetime
6. use Flask or FastAPI providers once request lifecycles matter
7. install `gemstone-py[fast]` only after the ctypes path is working

8. add benchmarks and live tests once the system is real

That order keeps the learning curve honest without making the first week feel like a database theology seminar.

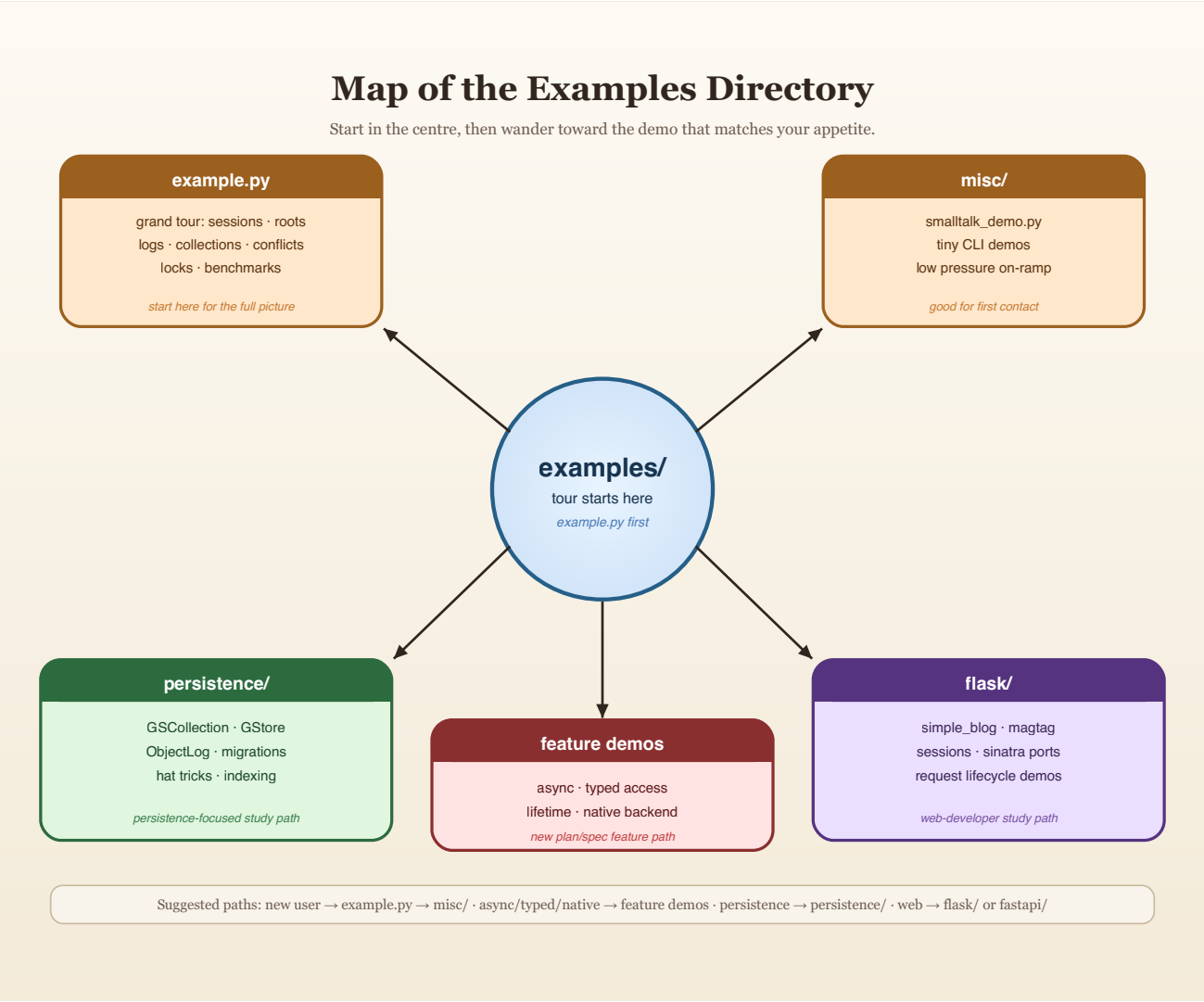
Manual Summary

If you remember only five things:

1. Be explicit about transaction policy.
2. Use `PersistentRoot` first unless you need something more specialized.
3. Use async wrappers for async frameworks, not for shared-session threading.
4. Use typed and managed OOP wrappers when raw integers hide too much intent.
5. The examples teach the package; the maintained workflows prove it.

Guide to the Examples

The `examples/` tree is one of the best parts of `gemstone-py`. It is broad enough to teach the package properly, but disciplined enough that the examples still map to the real API and current workflows.



How to Use This Guide

This guide is organized around what you are trying to learn:

- first contact with a live GemStone session
- async sessions and FastAPI request integration
- typed OOPs, typed queries, and export-set lifetime handles
- persistence helpers
- bulk root/dictionary access and explicit value conversion
- indexed collections and query-style access
- concurrency primitives
- Flask integration
- optional native backend selection
- web app translations
- maintained benchmarks versus teaching examples

If you want the shortest path:

1. run `examples/quickstart.py`
2. run `examples/example.py`
3. run `examples/misc/smalltalk_demo.py`
4. run one feature example from `async_features/`, `typed_access/`, or `lifetime/`
5. inspect one persistence example or web example

Common Setup

Most live examples expect the same GemStone environment:

```
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

`GS_STONE_NAME` is accepted as a stone-name alias when `GS_STONE` is absent. Setting both to the same value is harmless and keeps the examples, VS Code workbench, and database explorer aligned.

Run examples as modules from the repository root so imports resolve exactly as they do in CI:

```
cd /path/to/gemstone-py
source .venv/bin/activate
python -m examples.quickstart
python -m examples.misc.smalltalk_demo
python -m examples.async_features.session_root_and_collection
```

If your prompt is in the parent directory, for example `src %`, change into the `gemstone-py` checkout first. The installed `gemstone-py` wheel exposes `gemstone-examples`, `gemstone-fastapi-example`, and `gemstone-litestar-example` from any directory; the repository-only `examples.*` modules require the checkout root on `PYTHONPATH`.

For a fresh or shared stone, you can also audit and initialize the GemStone-side roots that the persistence helpers use:

```
gemstone-bootstrap --status
gemstone-bootstrap
```

That initializes `GStoreRoot` for `GStore`, `GSQueryRoot` for `GSCollection`, and a `GemstonePyBootstrapVersion` marker. It is safe to rerun because it does not replace existing roots.

For a normal installed package:

```
python -m pip install gemstone-py
```

For a source checkout with development tools:

```
python -m pip install -e .[dev]
```

For the optional native backend example:

```
python -m pip install "gemstone-py[fast]"
```

For FastAPI examples from an installed package:

```
python -m pip install "gemstone-py[fastapi]"
gemstone-fastapi-example --reload
```

For Django examples or applications:

```
python -m pip install "gemstone-py[django]"
```

For Litestar examples or applications:

```
python -m pip install "gemstone-py[litestar]"
gemstone-litestar-example --reload
```

For FastAPI examples from a source checkout:

```
python -m pip install -e ".[examples]"
python -m examples.fastapi.run --reload
```

For Litestar examples from a source checkout:

```
python -m pip install -e ".[examples]"
python -m examples.litestar.run --reload
```

For a compact map of the broader example set:

```
gemstone-examples list
gemstone-examples plan3-map
gemstone-examples value-converters
```

Running Examples From VS Code

The repository now includes `vscode-gemstone-py-workbench/`, a companion VS Code extension scaffold for the Python-facing workflow. It adds a GemStone Py sidebar with example runners, environment/backend checks, docs/PDF launchers, maintainer scripts, Jasper links, setup verification actions, and launcher plus webview commands for `python-gemstone-database-explorer`.

For generated wrappers, run `GemStone: Open Codegen Explorer`. It connects to the configured database explorer's `/codegen/*` API, browses live dictionaries/classes/protocols/methods/source, lets you select wrapper targets, previews generated files through `/codegen/preview`, diffs them against the configured output package, saves normalized selection JSON to `codegen-workbench.json`, and can run the Codegen check, generation, FastAPI demo, and live target probe from the same view.

Install it from the Visual Studio Marketplace:

```
code --install-extension unicompute.gemstone-py-workbench
```

Marketplace page: https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-py-workbench

During extension development:

```
cd vscode-gemstone-py-workbench
npm install
npm run compile
```

Then open the extension folder in VS Code and press `F5`. Configure the `gemstonePy.env` settings with the same `GS_*` values shown above before running live examples. Leave `gemstonePy.repoPath` empty when the current workspace is the `gemstone-py` checkout, and set `gemstonePy.explorerPath` to a local `python-gemstone-database-explorer` checkout when you want explorer commands.

`examples/quickstart.py`: The First Live Check

Start here when you want the shortest path from environment variables to a confirmed GemStone round trip.

```
python -m examples.quickstart
gemstone-examples quickstart
```

It does three things:

- connects with `GemStoneConfig.from_env()`
- verifies the session with `3 + 4`
- stores a small dictionary under `UserGlobals` through `PersistentRoot`

This is the example to copy when you are creating a new application skeleton. The other examples are cookbook entries for specific features.

`examples/cookbook/`

This directory is the stable cookbook index. The runnable examples keep their historical paths, and `examples/cookbook/README.md` points each topic to the right script, package, or installed command.

```
gemstone-examples list
gemstone-examples plan3-map
gemstone-examples value-converters
```

Use it when you know the kind of task you want but do not yet know which example directory owns it.

The `value-converters` entry point is intentionally offline. It previews the explicit converter registry and dataclass-to-dict helper without opening a live GemStone session, which makes it a useful first check for teams deciding how much conversion they want at application boundaries.

`examples/example.py`: The Grand Tour

If you only run one broad example after the quickstart, run this one.

Why it matters:

- it exercises the major package surfaces in one file
- it demonstrates live session behaviour instead of abstract promises
- it shows transaction boundaries, cross-session reads, and conflict patterns

It covers:

- `SmalltalkBridge`
- `GemStoneSessionFacade`
- `PersistentRoot`
- bulk `PersistentRoot` and selector-send helpers
- `GStore`
- `ObjectLog`
- `RCCounter`

- `RCHash`
- `RCQueue`
- nested transactions
- `CommitConflictError`
- date/time conversions
- object locking
- instance listing

Treat it as the "here is what the package can really do" demo.

Usage:

```
python -m examples.example
```

The core shape is still ordinary session code:

```
from gemstone_py import GemStoneConfig, GemStoneSession, TransactionPolicy
from gemstone_py.session_facade import GemStoneSessionFacade

config = GemStoneConfig.from_env()

with GemStoneSession(
    config=config,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
) as session:
    facade = GemStoneSessionFacade(session)
    facade["ExampleDict"] = {"status": "ok"}
```

`examples/misc/`

This is the low-pressure on-ramp.

Start here when:

- you want a tiny first success
- you want to confirm the environment is healthy
- you want a bridge demo without reading a hundred moving parts

The star here is `smalltalk_demo.py`, which now runs through the supported CLI surface and shows the Smalltalk bridge without dragging in everything else.

Usage:

```
python -m examples.misc.smalltalk_demo
gemstone-smalltalk-demo
gemstone-examples smalltalk-demo
```

Representative code:

```
from gemstone_py.example_support import MANUAL_POLICY, example_session
from gemstone_py.smalltalk_bridge import SmalltalkBridge

with example_session(transaction_policy=MANUAL_POLICY) as session:
    smalltalk = SmalltalkBridge(session)
    settings = smalltalk.StringKeyValueDictionary.new()
    settings["status"] = "ok"
    print(settings["status"])
```

`examples/async_features/`

This directory demonstrates the async API added for modern Python application stacks.

`session_root_and_collection.py` shows:

- `AsyncSession.connect(...)`
- `async with session.transaction()`
- `AsyncPersistentRoot`
- `AsyncManagedOop`
- `AsyncGSCollection`

Run it when the normal sync login works and you want to confirm that GCI work is being routed through the async facade correctly.

Usage:

```
python -m examples.async_features.session_root_and_collection
```

Session and root access:

```
from gemstone_py.aio import AsyncPersistentRoot, AsyncSession

async with AsyncSession.connect(config=config) as session:
    async with session.transaction():
        root = AsyncPersistentRoot(session)
        await root.set("AsyncExample", {"status": "async"})

    saved = await session.root().get("AsyncExample")
```

Async managed OOPs:

```
ref = await session.execute_managed("OrderedCollection new")
try:
    first = await session.new_string("from async")
    await ref.send("add:", first)
    print(await ref.print_string())
finally:
    await ref.close()
```

Async indexed collection access:

```
from gemstone_py.aio import AsyncGSCollection

collection = AsyncGSCollection("AsyncFeaturePeople", config=config)
await collection.bulk_insert([
    {"@name": "Ada", "@status": "active"},
    {"@name": "Grace", "@status": "inactive"},
])
await collection.add_index("@status")
active = await collection.search("@status", "eql", "active")
collection.close()
```

For larger async result sets, use the async iterator:

```
async for record in collection.search_iter("@status", "eql", "active",
chunk_size=500):
    print(record["@name"])
```

Use a generated or test-specific collection name in real scripts, and call `AsyncGSCollection.drop(...)` during cleanup.

`examples/typed_access/`

This directory covers the static typing stream from the plan.

Use it when you want to understand:

- `@gemstone_class(...)`
- `TypedOop[T]`
- `typed_oop(...)`
- typed `GSCollection.query(Protocol)` predicates
- chunked `Query.iter(...)` result handling
- `gemstone-codegen` generated wrappers for method-shaped Smalltalk access
- generated wrapper `.pyi` stubs and lightweight runtime metadata

`simple_blog_queries.py` contains importable helper functions for the blog data shape. `typed_oops_and_queries.py` is the runnable live example. `codegen_demo/` shows the Protocol-to-generated-wrapper workflow.

Usage:

```
python -m examples.typed_access.typed_oops_and_queries
python -m examples.typed_access.codegen_demo.preview
gemstone-codegen \
  --module examples.typed_access.codegen_demo.models \
  --output examples/typed_access/codegen_demo/generated \
  --check
python -m examples.typed_access.codegen_demo.run --reload
python -m examples.typed_access.codegen_demo.live_probe --booking-id B-1001
```

Typed OOPs:

```
from typing import Protocol
from gemstone_py import GemStoneSession, TransactionPolicy, gemstone_class

@gemstone_class("Date")
class GemStoneDate(Protocol):
    @property
    def printString(self) -> str:
        ...

with GemStoneSession(
    config=config,
    transaction_policy=TransactionPolicy.ABORT_ON_EXIT,
) as session:
    today = session.execute_typed("Date today", GemStoneDate)  # type: ignore[type-
abstract]
    print(today.gemstone_class_name)
    print(today.proxy().printString)
```

Typed `GSCollection` queries:

```
from typing import Protocol
from gemstone_py.gsquery import GSCollection

class BlogPostRecord(Protocol):
    title: str
    status: str

posts = GSCollection("SimplePosts", config=config)
published = posts.query(BlogPostRecord).where(
    lambda post: post.status == "published"
).all()
```

```
for post in published:
    print(post.title)
```

The lambda records a GemStone ivar path. It is not a Python-side row filter.

Generated wrappers:

```
from typing import Protocol
from gemstone_py import GemStoneConfig, GemStoneSession, gemstone_class,
gemstone_selector

@gemstone_class("OkzBooking")
class OkzBookingProto(Protocol):
    status: str

    @classmethod
    @gemstone_selector("findById:")
    def find_by_id(cls, booking_id: str) -> "OkzBookingProto":
        ...

from examples.typed_access.codegen_demo.generated import OkzBooking

with GemStoneSession(config=GemStoneConfig.from_env()) as session:
    booking = OkzBooking.find_by_id(session, "B-1001")
    print(booking.status)
```

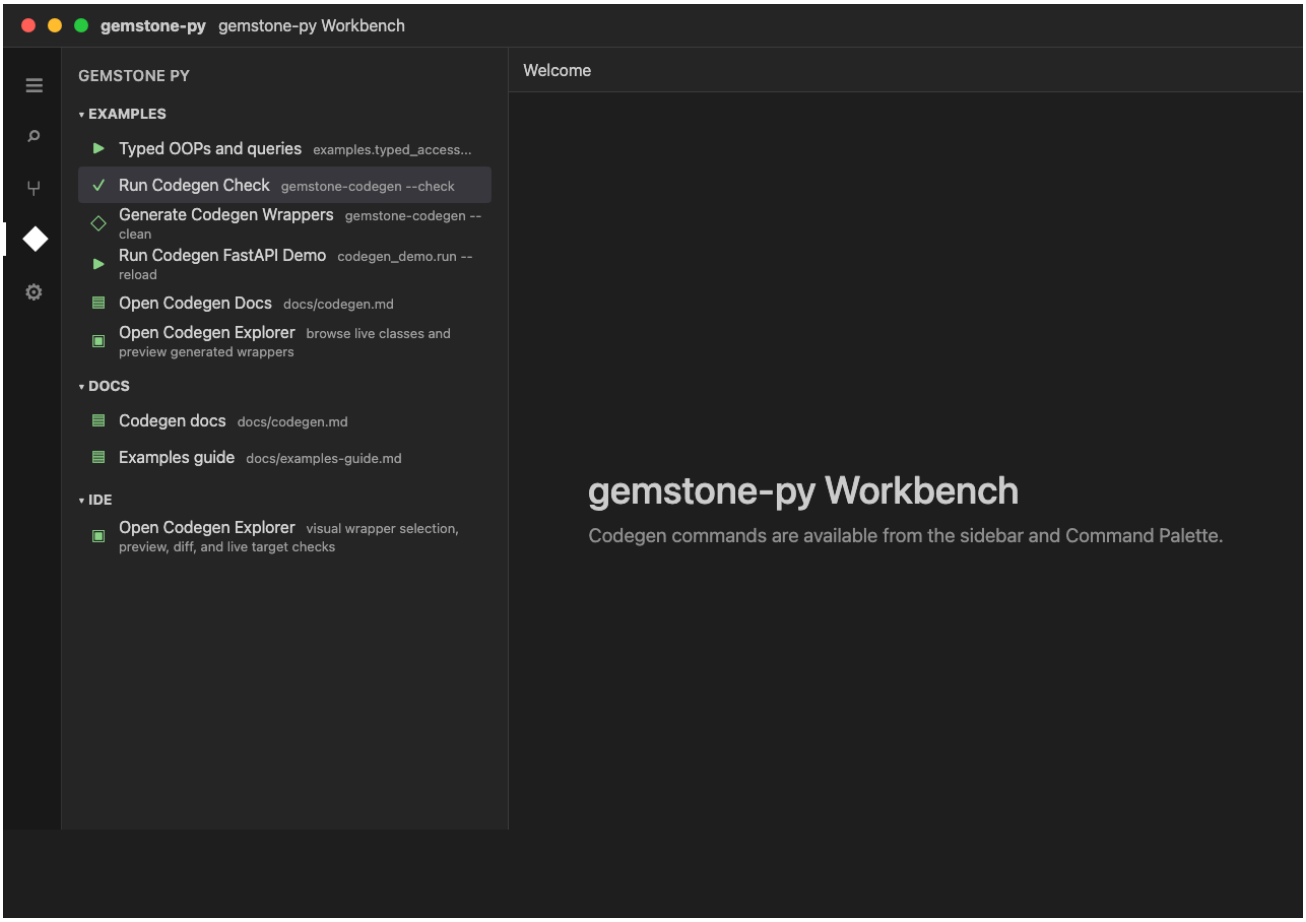
Use `@gemstone_selector(...)` for selectors that do not map cleanly from Python snake_case to Smalltalk camelCase keywords.

Generated wrappers expose `__gemstone_protocol__` and `__gemstone_selectors__`, so examples and tools can explain which Protocol and selector mapping produced a Python method without adding a runtime mapper.

The offline preview example is the safest place to start. It generates the `OkzBooking` and `OkzCustomer` wrappers in a temporary directory, shows the generated package layout, and prints the exact `gemstone-codegen --clean` and `--check` commands:

```
python -m examples.typed_access.codegen_demo.preview --show-source
```

The VS Code workbench can then make the live-class selection concrete:



Open `GemStone: Open Codegen Explorer`, connect to `python-gemstone-database-explorer`, select `OkzBooking` and `OkzCustomer`, preview the generated files, diff them against `examples/typed_access/codegen_demo/generated`, then save the selection. The example mapping file records the live-stone selection in the normalized selection JSON shape shared with the browser explorer. It includes fields, selectors, generated Python names, argument names, and return annotations:

```
examples/typed_access/codegen_demo/codegen-workbench.example.json
```

`examples/lifetime/`

`managed_oop_handles.py` is the focused example for the OOP lifetime model.

It demonstrates:

- `execute_managed(...)` for automatic export-set retention
- `ManagedOop.close()` for explicit cleanup
- `session.handle(raw_oop)` for scoped retention when you already have an OOP

Read this before holding raw OOP integers across operations that might trigger GemStone-side garbage collection.

Usage:

```
python -m examples.lifetime.managed_oop_handles
```

Managed handle:

```
collection = session.execute_managed("OrderedCollection new")
try:
    collection.send("add:", session.new_string("managed"))
    session.eval("System startGcAndCommit")
    print(collection.print_string())
finally:
    collection.close()
```

Scoped raw-OOP handle:

```
raw_oop = session.execute_oop("OrderedCollection new")
with session.handle(raw_oop) as handle:
    handle.send("add:", session.new_string("scoped"))
    print(handle.send("printString"))
```

Use `execute_managed(...)` for object references you want Python to retain automatically. Use `session.handle(...)` when an existing raw OOP only needs to be protected inside a known block.

examples/native_backend/

`check_backend.py` prints the selected low-level GCI backend and whether the optional native extension is importable.

Use it after:

```
python -m pip install "gemstone-py[fast]"
```

It is also useful when comparing benchmark reports with `GEMSTONE_PY_GCI_BACKEND=ctypes` and `GEMSTONE_PY_GCI_BACKEND=native`.

Usage:

```
python -m examples.native_backend.check_backend
GEMSTONE_PY_GCI_BACKEND=ctypes python -m examples.native_backend.check_backend
GEMSTONE_PY_GCI_BACKEND=native python -m examples.native_backend.check_backend
```

The code is deliberately tiny:

```
from gemstone_py import _gci
from gemstone_py.native import native_fast_path_available

print(_gci.IMPLEMENTATION)
print(native_fast_path_available())
```

Backend selection happens when Python imports `gemstone_py._gci`, so set `GEMSTONE_PY_GCI_BACKEND` before starting Python.

examples/persistence/

This is where the package stops being a client library and starts feeling like a practical application toolkit.

Major themes in this tree:

- indexed collections
- stores
- ordered collections and batch updates
- data migration
- persistent data structures
- translation of old MagLev-era ideas into plain GemStone use

Indexing

Files around `persistence/indexing/` show how to:

- create a dataset
- store rows in GemStone
- build indexes
- search efficiently

This is the right cluster to study when `PersistentRoot` starts to feel too coarse-grained for the shape of your data.

Usage:

```
python -m examples.persistence.indexing.create_random_people
python -m examples.persistence.indexing.search_random_people
python -m examples.persistence.indexing.index_example
```

The useful API shape:

```
from gemstone_py.gsquery import GSCollection
```

```
people = GSCollection("People", config=config)
people.bulk_insert([
    {"@name": "Ada", "@age": 36},
    {"@name": "Grace", "@age": 42},
])
people.add_index_for_class("@age", "SmallInt")
matches = people.search("@age", "gte", 40)
```

GStore

The `persistence/gstore/` example contrasts GemStone-backed storage with an in-memory dict baseline. That makes it useful for both teaching and performance intuition:

- you see the API shape
- you see the cost of persistence
- you do not need mythology to explain the result

Usage:

```
python -m examples.persistence.gstore.main
```

Typical store-shaped code:

```
from gemstone_py.gstore import GStore

store = GStore("inventory.db", config=config)
store["sku:hat"] = {"name": "Hat", "stock": 8}
print(store["sku:hat"])
```

Hat Trick

The `hat_trick/` example is the kind of thing every useful repository-backed toolkit should have: a weird little demo that is memorable enough to teach the real abstraction.

Here the abstraction is queue-backed shared state:

- create a hat backed by `RCQueue`
- inspect the contents
- understand the relation between a playful example and a real concurrency primitive

The queue is not a toy. The hat is merely wearing formal attire.

Usage:

```
python -m examples.persistence.hat_trick.create_hat
python -m examples.persistence.hat_trick.add_rabbit_to_hat
python -m examples.persistence.hat_trick.show_hat_contents
```

The pattern behind the example is reduced-conflict queue-backed state:

```
from gemstone_py.concurrency import RCQueue
from gemstone_py.persistent_root import PersistentRoot

root = PersistentRoot(session)
root["WorkQueue"] = RCQueue(session)
root["WorkQueue"].push("first job")
```

Migrations

The migrations examples are valuable because they show the package in an honest "old data must become new data" mode.

Use these when you want patterns for:

- versioned domain objects
- chunked migration
- retry loops
- explicit transactional migration steps
- module-style forward and rollback migrations

Usage:

```
python -m examples.persistence.migrations.write_posts
python -m examples.persistence.migrations.migrate
python -m examples.persistence.migrations.migrate_by_chunks
```

For application migrations, scaffold a numbered file:

```
gemstone-migrations scaffold add_amount_to_booking --directory migrations
```

That creates a module with `upgrade(session)` and `downgrade(session)`. Register files in a manifest:

```
migrations = [
    "my_app.migrations.001_initial",
    "my_app.migrations.002_add_amount_to_booking",
]
```

Then apply or dry-run the manifest:

```
from gemstone_py import GemStoneConfig, GemStoneSession
from gemstone_py.migrations import current_version, load_manifest, upgrade

steps = load_manifest("my_app.migrations.manifest")

with GemStoneSession(config=GemStoneConfig.from_env()) as session:
    print(current_version(session))
    print(upgrade(session, steps, dry_run=True))
    upgrade(session, steps)
```

Example migration body:

```
id = "002_add_amount_to_booking"
dependencies = ("001_initial",)

def upgrade(session):
    session.eval(
        "OkzBooking addInstVarName: 'amount' ifAbsent: [ nil ]"
    )
    session.commit()

def downgrade(session):
    session.eval(
        "OkzBooking removeInstVarName: 'amount' ifAbsent: [ nil ]"
    )
    session.commit()
```

The same workflow is available from the command line:

```
gemstone-migrations current
gemstone-migrations status --manifest my_app.migrations.manifest
gemstone-migrations plan --manifest my_app.migrations.manifest
gemstone-migrations upgrade --manifest my_app.migrations.manifest --dry-run
gemstone-migrations upgrade --manifest my_app.migrations.manifest --dry-run --record
gemstone-migrations upgrade --manifest my_app.migrations.manifest
gemstone-migrations downgrade --manifest my_app.migrations.manifest --target
001_initial
```

Before applying a new step, the runner checks that every GemStone-applied migration id still exists in the local manifest and that stored checksums still match local migration files when checksums are available.

Real `upgrade` and `downgrade` runs also acquire an advisory lock in `UserGlobals` before applying steps. Dry-runs do not acquire the lock. If a previous process crashed and left a stale lock, use a reviewed override:

```
gemstone-migrations upgrade --manifest my_app.migrations.manifest \
  --force-lock --lock-owner release-2026-05-11
```

Use `--no-lock` only for controlled maintenance windows where another guard is already preventing concurrent migration runs.

Plain `--dry-run` prints the planned migration ids. Add `--record` when the migration body is mostly direct session calls such as `session.eval(...)`; the runner executes callbacks against a recording session and prints the calls it would have made without sending them to GemStone.

Recorded output is intentionally plain so it can be reviewed in a release log:

```
dry-run upgrade: 1 step(s)
  002_add_amount_to_booking
recorded operations:
  # upgrade 002_add_amount_to_booking
  session.eval("OkzBooking addInstVarName: 'amount' ifAbsent: [ nil ]")
  session.commit()
```

If a migration branches on live query results, use the recorded dry-run as a partial review aid only. The recorder returns harmless placeholders for OOPs and does not ask GemStone for real state.

When a local Protocol or type witness has drifted from a GemStone class, compare the two before writing a migration:

```
gemstone-migrations diff-class OkzBooking \
  --local-class my_app.models:OkzBookingProto
```

The command prints missing and extra instance variables plus advisory `session.eval(...)` lines that can be copied into a reviewed migration.

For startup or release checks, fingerprint the expected roots, classes, and migration state:

```
gemstone-migrations fingerprint \
  --root Bookings \
  --class OkzBooking \
  --manifest my_app.migrations.manifest \
  --json
```

The key habit is to keep migration units explicit:

```
from gemstone_py import session_scope

with session_scope(config=config) as session:
    # read old shape
    # write new shape
```

```
    # commit happens only if the block succeeds
    ...
```

examples/flask/

This directory is where the request/session layer stops being theory.

Important sub-examples:

- `simple_blog/`
- `magtag/`
- `sessions/`
- `sinatra_port/`

simple_blog

Good first Flask example because:

- the domain is small
- the persistence is recognizable
- the routes are ordinary enough to reason about quickly

Study this when you want:

- request session handling
- a simple persistent model
- proof that the package fits normal Flask structure

Usage:

```
python -m examples.flask.simple_blog.blog_app
```

Request-scoped work should use the installed session integration:

```
from flask import Flask
from gemstone_py import GemStoneConfig, install_flask_request_session

app = Flask(__name__)
install_flask_request_session(
    app,
    config=GemStoneConfig.from_env(),
    pool_size=4,
)
```

magtag

This is a larger demonstration of the same core idea:

- Flask app
- GemStone-backed data
- `GSCollection` in a realistic setting

If `simple_blog` is the friendly coffee, `magtag` is the "all right, show me a real screen and some real workflows" example.

Usage:

```
python -m examples.flask.magtag.magtag_app
```

The model layer uses `GSCollection` for lookup-heavy records:

```
from gemstone_py.gsquery import GSCollection

users = GSCollection("MagTagUsers")
users.add_index_for_class("@name", "String")
matches = users.search("@name", "eql", "pbm0")
```

sessions

These examples focus on request/session lifecycle concerns. They are useful when you care more about integration behaviour than about application domain logic.

Usage:

```
python -m examples.flask.sessions.msessions
```

Provider health can be inspected without reaching into private Flask state:

```
from gemstone_py import flask_request_session_provider_snapshot

snapshot = flask_request_session_provider_snapshot(app)
if snapshot is not None:
    print(snapshot.available, snapshot.in_use, snapshot.created)
```

examples/fastapi/

This is the smallest async web example. It uses `session_dependency(...)` to open one `AsyncSession` per request and commit or abort with the request outcome.

Usage:

```
python -m pip install -e ".[examples]"
python -m examples.fastapi.run --reload
```

When the server starts, you should see output like:

```
INFO:     Will watch for changes in these directories: ['/path/to/gemstone-py']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [49045] using WatchFiles
INFO:     Started server process [49048]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

With that server running, test it from a second terminal.

Basic checks:

```
curl -i http://127.0.0.1:8000/
```

Expected:

```
HTTP/1.1 200 OK
```

Body should include:

```
{"name":"gemstone-py FastAPI example","endpoints":{"health":"/health/
gemstone","docs":"/docs","openapi":"/openapi.json"}}
```

Then test the GemStone endpoint:

```
curl -i http://127.0.0.1:8000/health/gemstone
```

Expected if GemStone credentials/environment are set and the stone is reachable:

```
{"result":7}
```

Also open these in a browser:

```
http://127.0.0.1:8000/
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/health/gemstone
```

Endpoint shape:

```
from fastapi import Depends, FastAPI
from gemstone_py import GemStoneConfig
from gemstone_py.aio import AsyncSession, AsyncSessionPool
from gemstone_py.aio.fastapi import pool_session_dependency, session_dependency

app = FastAPI()
get_gemstone_session = session_dependency(config=GemStoneConfig.from_env())

@app.get("/health/gemstone")
async def gemstone_health(session: AsyncSession = Depends(get_gemstone_session)):
    return {"result": await session.eval("3 + 4")}
```

For production-style request reuse, use a pool-backed dependency:

```
pool = AsyncSessionPool(maxsize=8, minsize=2, config=GemStoneConfig.from_env())
get_gemstone_session = pool_session_dependency(pool)
```

`examples/litestar/`

This is the smallest Litestar web example. It uses `gemstone_py.aio.litestar.session_dependency(...)` to open one `AsyncSession` per request and finalize it with the request outcome.

Usage:

```
python -m pip install -e ".[examples]"
python -m examples.litestar.run --reload
```

The root route returns the available endpoints:

```
{"name":"gemstone-py Litestar
example","framework":"Litestar","adapter":"gemstone_py.aio.litestar.session_dependenc
y","dependencyInjection":"litestar.di.Provide","endpoints":{"health":"/health/
gemstone","docs":"/schema/swagger","openapi":"/schema/openapi.json"}}
```

This deliberately mirrors the FastAPI example's `/` and `/health/gemstone` contract while showing the Litestar-specific adapter, `Provide(...)` dependency injection, and schema routes.

The installed package runner is:

```
gemstone-litestar-example --reload
```

`examples/django/` and `examples/webstack/`

These matter for two reasons:

1. they prove the package is not locked into one tiny application style
2. they surface compatibility issues that smaller demos never trigger

The `webstack` examples are also useful for release verification because they have enough app surface to catch missing imports, route regressions, and other "this only broke in CI" problems.

Usage:

```
cd examples/django/myapp
python manage.py migrate
python manage.py runserver
```

```
python -m examples.webstack.magtag_app
```

Examples vs Maintained Lanes

This distinction matters.

The examples are for learning and exploration.

The maintained lanes are:

- `./scripts/run_ci_checks.sh`
- `./scripts/run_live_checks.sh`
- `gemstone-benchmarks`
- the GitHub workflows under `.github/workflows/`

Do not confuse:

- "I ran a charming example"
- with
- "the package is verified against its supported workflows"

You need both. They are not interchangeable.

Suggested Study Paths

Path 1: New user

1. `examples/quickstart.py`
2. `examples/example.py`
3. `examples/misc/smalltalk_demo.py`
4. this guide again, now with less fear

Path 2: Persistence-focused user

1. `examples/quickstart.py`
2. `examples/example.py`
3. `examples/persistence/indexing/*`
4. `examples/persistence/gstore/main.py`
5. `examples/persistence/migrations/*`

Path 3: Async or FastAPI user

1. `examples/async_features/session_root_and_collection.py`
2. `examples/fastapi/app.py`
3. `tests/test_live_async_integration.py`

Path 4: Typed API user

1. `examples/typed_access/typed_oops_and_queries.py`
2. `examples/typed_access/simple_blog_queries.py`
3. `tests/test_typed_oop_lifetime.py`
4. `tests/test_gsquery.py`

Path 5: Web developer

1. `examples/flask/simple_blog/*`
2. `examples/flask/sessions/*`
3. `examples/flask/magtag/*`
4. `examples/fastapi/app.py`
5. `examples/webstack/*`

Path 6: Concurrency-curious person

1. `examples/example.py`
2. hat trick queue demo
3. live tests around contention and retries

Path 7: Packaging and performance user

1. `examples/native_backend/check_backend.py`
2. `gemstone-benchmarks --suite gci --entries 1000000`
3. `docs/performance.md`
4. `./scripts/run_native_checks.sh`
5. `GS_RUN_LIVE=1 python scripts/run_native_rust_core_live_smoke.py --require-live`

Practical Advice

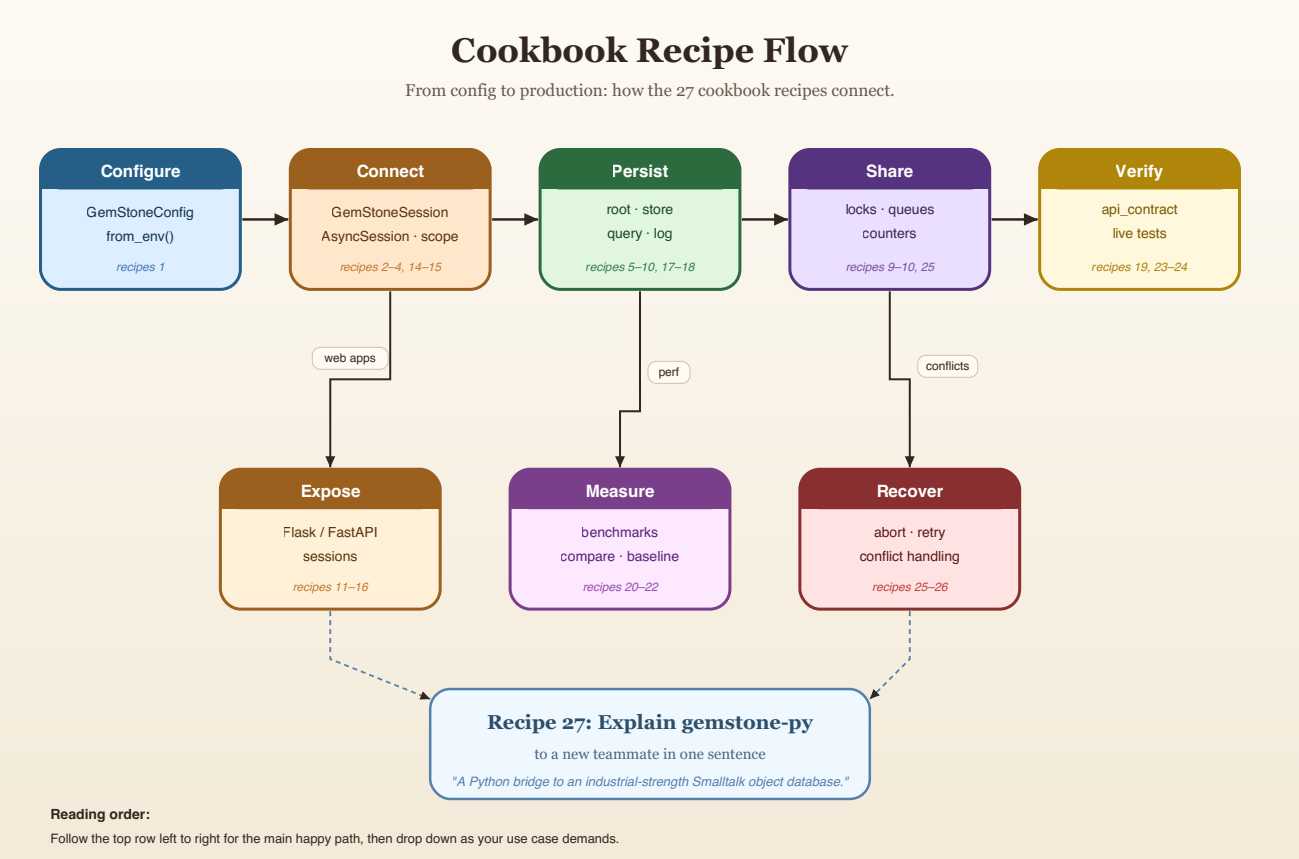
- run the examples from a configured environment, not from a shell that "mostly remembers" the right variables
- do not start with the biggest web example if you still do not know how `TransactionPolicy` works
- read the maintained benchmark docs before treating example performance output as policy
- keep the examples open next to the user manual; the two reinforce each other

Closing Thought

The examples directory is not filler. It is one of the package's strongest teaching tools, and it is worth reading like source documentation instead of like a pile of promotional demos.

Cookbook

This cookbook is a collection of direct recipes. Each one is deliberately short enough to copy, adapt, and keep moving.



Recipe 1: Open a Session and Evaluate Smalltalk

```
from gemstone_py import GemStoneConfig, GemStoneSession

config = GemStoneConfig.from_env()

with GemStoneSession(config=config) as session:
    print(session.eval("3 factorial"))
```

Use this when:

- you want the smallest live sanity check
- you need to inspect a repository value quickly

Recipe 2: Commit a Write Safely

```
from gemstone_py import GemStoneConfig, TransactionPolicy, GemStoneSession
from gemstone_py.persistent_root import PersistentRoot

config = GemStoneConfig.from_env()

with GemStoneSession(
```

```
    config=config,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
) as session:
    root = PersistentRoot(session)
    root["CookbookExample"] = {"answer": 42}
```

Why this recipe exists:

- it avoids the classic "the write seemed to work but vanished" mistake

Recipe 3: Read From `PersistentRoot`

```
from gemstone_py import GemStoneSession
from gemstone_py.persistent_root import PersistentRoot

with GemStoneSession(config=config) as session:
    root = PersistentRoot(session)
    print(root["CookbookExample"])
```

Recipe 4: Use `session_scope(...)`

```
from gemstone_py import GemStoneConfig, session_scope
from gemstone_py.persistent_root import PersistentRoot

config = GemStoneConfig.from_env()

with session_scope(config=config) as session:
    root = PersistentRoot(session)
    root["ScopedWork"] = {"kind": "unit-of-work"}
```

This is the recipe to prefer in application code.

Recipe 5: Create a Named `GSCollection`

```
from gemstone_py.gsquery import GSCollection

people = GSCollection("CookbookPeople", config=config)
people.insert({"@name": "Ada", "@city": "London"})
people.insert({"@name": "Grace", "@city": "New York"})
people.add_index("@city")
```

Recipe 6: Search an Indexed Collection

```
matches = people.search("@city", "eql", "London")
for row in matches:
    print(row)
```

Use `GSCollection` when you need repeated search, not when you merely enjoy the idea of repeated search.

Recipe 7: Keep a Store-Like Dataset in `GStore`

```
from gemstone_py.gstore import GStore

store = GStore("cookbook.db", config=config)
store["sku:hat"] = {"name": "Hat", "stock": 8}
store["sku:cape"] = {"name": "Cape", "stock": 3}
print(store["sku:hat"])
```

Recipe 8: Append a Persistent Event Log Entry

```
from gemstone_py.objectlog import ObjectLog

log = ObjectLog(config=config)
log.info("inventory_adjusted", {"sku": "hat", "delta": -1})
```

Later:

```
for entry in log.entries():
    print(entry)
```

Recipe 9: Share a Counter Between Sessions

```
from gemstone_py.concurrency import RCCounter

with session_scope(config=config) as session:
    counter = RCCounter(session)
    counter += 1
```

Use shared counters when the number truly lives in GemStone, not when a local integer would do and you are simply feeling theatrical.

Recipe 10: Share a Queue

```
from gemstone_py.concurrency import RCQueue
from gemstone_py.persistent_root import PersistentRoot

with session_scope(config=config) as session:
    root = PersistentRoot(session)
    root["WorkQueue"] = RCQueue(session)
```

Recipe 11: Install Flask Request Sessions

```
from flask import Flask
from gemstone_py import GemStoneConfig
from gemstone_py.frameworks.flask import install_flask_request_session

app = Flask(__name__)

install_flask_request_session(
    app,
    config=GemStoneConfig.from_env(),
    pool_size=4,
)
```

This is the default production-friendly shape.

Recipe 12: Use a Thread-Local Provider for Simpler Hosting

```
install_flask_request_session(
    app,
    config=GemStoneConfig.from_env(),
    thread_local=True,
)
```

Good when the server model is simple and you want one session per thread.

Recipe 13: Inspect Provider Health

```
from gemstone_py import flask_request_session_provider_snapshot

@app.get("/health/gemstone")
```

```
def gemstone_health():
    return flask_request_session_provider_snapshot()
```

Operational visibility is better than optimism.

Recipe 14: Build a Custom Web Adapter

```
from gemstone_py import GemStoneConfig, RequestScope, TransactionPolicy
from gemstone_py.session_providers import GemStoneSessionPool

pool = GemStoneSessionPool(maxsize=4, config=GemStoneConfig.from_env())

scope = RequestScope(
    session_provider=pool,
    transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
)
session = scope.session()
try:
    session.eval("3 + 4")
finally:
    scope.finalize()
```

`RequestScope` is the framework-neutral sync lifecycle primitive behind Flask request sessions. For async frameworks, pair `AsyncRequestScope` with `AsyncSessionPool` and await `scope.finalize(...)` from request cleanup.

Recipe 15: Open an Async Session

```
from gemstone_py import GemStoneConfig
from gemstone_py.aio import AsyncSession

config = GemStoneConfig.from_env()

async with AsyncSession.connect(config=config) as session:
    print(await session.eval("3 + 4"))
```

Use this when the surrounding application is already async.

Recipe 16: Use an Async Transaction

```
async with AsyncSession.connect(config=config) as session:
    async with session.transaction():
```

```
        root = session.root()
        await root.set("AsyncCookbook", {"status": "ok"})
```

The transaction context commits on success and aborts on exception.

Recipe 17: Add FastAPI Dependency Injection

```
from fastapi import Depends, FastAPI
from gemstone_py import GemStoneConfig
from gemstone_py.aio import AsyncSession
from gemstone_py.aio.fastapi import session_dependency

app = FastAPI()
get_gemstone = session_dependency(config=GemStoneConfig.from_env())

@app.get("/health/gemstone")
async def gemstone_health(session: AsyncSession = Depends(get_gemstone)):
    return {"result": await session.eval("3 + 4")}
```

Django apps use the sync middleware adapter:

```
from django.http import JsonResponse
from gemstone_py import GemStoneConfig
from gemstone_py.frameworks.django import GemStoneSessionMiddleware, request_session

def gemstone_session_middleware(get_response):
    return GemStoneSessionMiddleware(get_response, config=GemStoneConfig.from_env())

def gemstone_health(request):
    session = request_session(request)
    return JsonResponse({"result": session.eval("3 + 4")})
```

For Litestar, use the sibling async adapter:

```
from litestar import Litestar, get
from litestar.di import Provide
from gemstone_py import GemStoneConfig
from gemstone_py.aio.litestar import session_dependency

get_gemstone = session_dependency(config=GemStoneConfig.from_env())

@get("/health/gemstone", dependencies={"session": Provide(get_gemstone)})
async def gemstone_health(session):
    return {"result": await session.eval("3 + 4")}

app = Litestar(route_handlers=[gemstone_health])
```

Recipe 18: Query With a Typed Protocol

```
from typing import Protocol
from gemstone_py.gsquery import GSCollection

class PostRecord(Protocol):
    title: str
    status: str

posts = GSCollection("SimplePosts", config=config).query(PostRecord)
for post in posts.where(lambda row: row.status == "published").all():
    print(post.title)
```

The lambda records an indexed GemStone path. It is not called for every row in Python.

Recipe 19: Generate a Typed Smalltalk Wrapper

```
from typing import Protocol
from gemstone_py import gemstone_class, gemstone_selector

@gemstone_class("OkzBooking", async_=True)
class OkzBookingProto(Protocol):
    status: str

    @classmethod
    @gemstone_selector("findById:")
    def find_by_id(cls, booking_id: str) -> "OkzBookingProto":
        ...

    def mark_paid(self, at_posix_seconds: int) -> None:
        ...
```

Generate and check in the wrapper:

```
gemstone-codegen \
  --module examples.typed_access.codegen_demo.models \
  --output examples/typed_access/codegen_demo/generated \
  --clean
```

Use it without hand-writing the class-side Smalltalk string:

```
from examples.typed_access.codegen_demo.generated import OkzBooking

booking = OkzBooking.find_by_id(session, "B-1001")
```

```
print(booking.status)
booking.mark_paid(1_779_912_000)
```

Recipe 20: Keep an OOP Alive Explicitly

```
with GemStoneSession(config=config) as session:
    ref = session.execute_managed("OrderedCollection new")
    ref.send("add:", session.new_string("kept"))
    print(ref.print_string())
    ref.close()
```

For a raw OOP:

```
with session.handle(raw_oop) as handle:
    print(handle.send("printString"))
```

Recipe 21: Check the Native Backend

```
python -m pip install "gemstone-py[fast]"
python -m examples.native_backend.check_backend
```

Force a backend before Python starts:

```
GEMSTONE_PY_GCI_BACKEND=ctypes python -m examples.native_backend.check_backend
GEMSTONE_PY_GCI_BACKEND=native python -m examples.native_backend.check_backend
```

Recipe 22: Run the Maintained Benchmarks

```
gemstone-benchmarks --entries 500 --search-runs 20
```

Or emit JSON:

```
gemstone-benchmarks --json --output benchmark-report.json
```

Recipe 23: Compare Benchmark Reports

```
gemstone-benchmark-compare old.json new.json --json --output compare.json
```

That turns performance arguments into evidence, which is disappointingly healthy.

Recipe 24: Register a New Accepted Benchmark Baseline

```
gemstone-benchmark-baseline-register \
  benchmark-report.json \
  --manifest .github/benchmarks/index.json
```

Recipe 25: Verify the Installed Artifact

```
python -m gemstone_py.api_contract --json
```

This is useful after installation, after release, and after any moment when you feel your package metadata may have become sentient.

For a full PyPI/TestPyPI release check:

```
gemstone-publish-verify --gemstone-version 0.2.12 --native-version 0.1.2
```

That command checks project JSON, version-specific JSON, the simple index, and temporary-virtualenv installs for both the pure package and native package.

Recipe 26: Scaffold and Run Module-Style Migrations

Create a numbered migration file:

```
gemstone-migrations scaffold add_amount_to_booking --directory migrations
```

A migration module exposes `upgrade(session)`, optional `downgrade(session)`, and optional `dependencies`:

```
id = "002_add_amount_to_booking"
dependencies = ("001_initial",)

def upgrade(session):
    session.eval(
        "OkzBooking addInstVarName: 'amount' ifAbsent: [ nil ]"
    )
    session.commit()

def downgrade(session):
    session.eval(
        "OkzBooking removeInstVarName: 'amount' ifAbsent: [ nil ]"
    )
    session.commit()
```

Register modules in a manifest and apply them:

```
from gemstone_py import GemStoneConfig, GemStoneSession
from gemstone_py.migrations import current_version, load_manifest, upgrade

steps = load_manifest("my_app.migrations.manifest")

with GemStoneSession(config=GemStoneConfig.from_env()) as session:
    print(current_version(session))
    print(upgrade(session, steps, dry_run=True))
    upgrade(session, steps)
```

Applied versions are recorded under `GemstonePyMigrations` in `UserGlobals`. The runner refuses to continue when the stone has applied migrations that are missing from the local manifest, or when a stored migration checksum differs from the local file. Real `upgrade` and `downgrade` runs acquire an advisory lock in `UserGlobals` before applying steps; dry-runs do not.

The same operations are available from the command line:

```
gemstone-migrations current
gemstone-migrations status --manifest my_app.migrations.manifest
gemstone-migrations plan --manifest my_app.migrations.manifest
gemstone-migrations upgrade --manifest my_app.migrations.manifest --dry-run
gemstone-migrations upgrade --manifest my_app.migrations.manifest --dry-run --record
gemstone-migrations upgrade --manifest my_app.migrations.manifest
gemstone-migrations downgrade --manifest my_app.migrations.manifest --target
001_initial
```

For a stale lock left by a crashed process, use a deliberate override:

```
gemstone-migrations upgrade --manifest my_app.migrations.manifest \
  --force-lock --lock-owner release-2026-05-11
```

`--dry-run --record` uses a recording session for callbacks and prints common session calls such as `session.eval(...)`, `execute(...)`, `perform_value(...)`, `commit()`, and `abort()` without sending them to GemStone.

Example recorded output:

```
dry-run upgrade: 1 step(s)
  002_add_amount_to_booking
recorded operations:
  # upgrade 002_add_amount_to_booking
  session.eval("OkzBooking addInstVarName: 'amount' ifAbsent: [ nil ]")
  session.commit()
```

Treat recorded dry-runs as a release review aid. Migrations that depend on live query results still need a real staging run because the recorder returns placeholder OOPs and never reads from GemStone.

To compare a local Protocol or type witness with the live GemStone class before writing the next migration:

```
gemstone-migrations diff-class OkzBooking \
  --local-class my_app.models:OkzBookingProto
```

The output lists missing and extra instance variables and prints advisory `session.eval(...)` lines for a reviewed migration.

Recipe 27: Run the Live Test Lane

```
GS_RUN_LIVE=1 ./scripts/run_live_checks.sh
```

Longer soak run:

```
GS_RUN_LIVE=1 GS_RUN_LIVE_SOAK=1 ./scripts/run_live_checks.sh
```

Recipe 28: Handle Commit Conflicts Without Pretending They Are Rare

When multiple sessions modify overlapping state, conflicts are normal. The right pattern:

```
from gemstone_py import GemStoneConfig, PersistentRoot, retrying_transaction
```

```
config = GemStoneConfig.from_env()

def increment_counter(session):
    # reload data each attempt; previous reads are stale after abort
    root = PersistentRoot(session)
    root["counter"] = (root.get("counter") or 0) + 1
    return root["counter"]

value = retrying_transaction(increment_counter, config=config, attempts=5)
```

Rules:

- keep the write unit small
- reload data after every abort — the previous read is stale
- bound the retry count — do not loop forever
- log conflicts — frequent conflicts signal a design smell, not bad luck

Add `on_conflict=` when you want a readable report for each retry:

```
def log_retry(retry):
    print(retry.format(include_summaries=False))

retrying_transaction(
    increment_counter,
    config=config,
    attempts=5,
    on_conflict=log_retry,
)
```

Recipe 29: Learn a Queue With a Hat

The hat trick example is memorable because it teaches a real primitive through a slightly ridiculous scenario. You should keep more examples like that in your own codebase than you probably do.

Recipe 30: Explain `gemstone-py` to a New Teammate

Use this sentence:

> "It is a Python package that talks directly to GemStone Smalltalk, keeps transactions explicit, gives us persistence helpers instead of a fake ORM, supports async, typed access, managed OOP lifetimes, and optional native wheels, and already has real CI, release, benchmark, and live verification lanes."

That sentence has rescued several meetings already. At minimum, it should rescue yours.

Recipe 31: Convert Scalar Values Explicitly

```
from datetime import date
from decimal import Decimal
from gemstone_py import GemStoneConfig, GemStoneSession,
scalar_value_converter_registry

registry = scalar_value_converter_registry()

with GemStoneSession(config=GemStoneConfig.from_env()) as session:
    invoice_oop = session.eval_oop("UserGlobals at: #CurrentInvoice")
    due_on, amount = registry.to_oops(session, [date(2026, 5, 15), Decimal("19.95")])
    session.perform_oop(invoice_oop, "dueOn:amount:", due_on, amount)
```

The built-in factories cover `datetime`, exact `date`, `Decimal`, and `UUID`. `ValueConverterRegistry.copy()` and `extend(...)` make application-specific registries cheap to assemble without global state.

For dataclasses, convert to a plain payload when that is what the GemStone-side API expects:

```
from gemstone_py import dataclass_to_dict

payload = dataclass_to_dict(booking_patch, recurse=False)
root["PendingBookingPatch"] = payload
```

Run the offline preview without a live stone:

```
gemstone-examples value-converters
python -m examples.cookbook.value_converters
```

Recipe 32: Batch Root and Dictionary Access

```
from gemstone_py import GemStoneConfig, PersistentRoot, session_scope

config = GemStoneConfig.from_env()

with session_scope(config=config) as session:
    root = PersistentRoot(session)

    values = root.get_many(["Settings", "VisitCount"], default=None)
    root.update_many({
        "VisitCount": int(values["VisitCount"] or 0) + 1,
        "LastMaintenance": "2026-05-14T22:00:00Z",
    })

    if "Settings" not in root:
```

```
        root["Settings"] = {}
    settings = root["Settings"]
    settings.update_many({"theme": "dark", "page_size": 50})
```

Use this when the operation is naturally a batch. It is still explicit repository access, just less chatty.

For persistent ordered lists:

```
from gemstone_py import PersistentRoot, session_scope
from gemstone_py.ordered_collection import OrderedCollection

with session_scope(config=config) as session:
    states = OrderedCollection(session)
    states.extend(["draft", "review", "paid"])
    PersistentRoot(session)["InvoiceStates"] = states
```

Recipe 33: Send Selectors in Bulk

```
from gemstone_py import GemStoneConfig, GemStoneSession, PerformCall

with GemStoneSession(config=GemStoneConfig.from_env()) as session:
    orders = [
        session.global_get("OrderA"),
        session.global_get("OrderB"),
    ]

    statuses = session.perform_many_value(orders, "status")
    labels = session.bulk_perform_calls_value([
        PerformCall(orders[0], "printString"),
        (orders[1], "status"),
    ])
```

Use `perform_many_` *for one selector across many receivers. Use* `bulk_perform_calls_` when the receivers, selectors, or arguments differ.

Recipe 34: Fingerprint Expected Schema

```
gemstone-migrations fingerprint \
  --root Bookings \
  --root BookingIndexes \
  --class OkzBooking \
  --manifest my_app.migrations.manifest \
  --json
```

For startup checks:

```
from gemstone_py.migrations import assert_schema_fingerprint, load_manifest

steps = load_manifest("my_app.migrations.manifest")
assert_schema_fingerprint(
    session,
    expected_digest="...",
    root_keys=["Bookings", "BookingIndexes"],
    class_names=["OkzBooking"],
    migration_steps=steps,
)
```

Fingerprinting is for deployment confidence. It does not apply migrations or infer Python object mappings.

Plan3 Feature Map

This page maps the plan3 improvement streams to the code, examples, and docs that now carry each feature. It is a quick orientation page for release review, new contributors, and the VS Code workbench docs view.

The same map is available from the command line:

```
gemstone-examples plan3-map
python -m examples.cookbook.plan3_feature_map
```

Status

Stream	Status	Main Surface
1. Pool + health	Landed	`GemStoneSessionPool`, `AsyncSessionPool`, pool-backed FastAPI/ Litestar dependencies
2. Streaming results	Landed	`GSCollection.search_iter(...)`, `GSCollection.iter(...)`, `Query.iter(...)`, async query iteration
3. Typed codegen	Landed	`gemstone-codegen`, generated wrappers, async wrappers, `.pyi` stubs, drift checks
4. Observability	Landed	tracing, metrics, slow-operation logging, pool/session observations
5. Migrations	Landed	module migrations, rollback, dry-run, class diff, schema fingerprinting

Stream	Status	Main Surface
6. Inspect/debug	Landed	`session.inspect(...)`, `session.dump(...)`, `session.describe_class(...)`, `gemstone-inspect`
7. Framework adapters	Landed	`web_core`, Flask/Django adapters, FastAPI/Litestar async adapters
8. Examples	Landed	quickstart, examples guide, cookbook map, workbench example runner
9. Performance docs	Landed	benchmark CLI, compare/register helpers, committed baseline docs
10. Native wheels	Landed	PyO3 native package, stable-ABI wheel workflow, release checks
11. Bootstrap	Landed	packaged GemStone-side `init.st`, `gemstone-bootstrap`, setup docs

Stream Details

Stream	Modules	Examples	Docs
Pool + health	`gemstone_py.session_providers`, `gemstone_py.aio.pool`	`examples/fastapi/`, `examples/litestar/`	User Manual, Cookbook
Streaming results	`gemstone_py.gsquery`, `gemstone_py.aio.gsquery`	`examples/async_features/`	Examples Guide, Performance
Typed codegen	`gemstone_py.codegen`	`examples/typed_access/codegen_demo/`	Type-Safe Smalltalk Codegen
Observability	`gemstone_py.observability`	`examples/fastapi/`, `examples/litestar/`	Observability
Migrations	`gemstone_py.migrations`	`examples/persistence/migrations/`	Cookbook, User Manual
Inspect/debug		`examples/cookbook/`	User Manual

Stream	Modules	Examples	Docs
	`gemstone_py.inspection`, `GemStoneSession.inspect`, `GemStoneSession.dump`, `GemStoneSession.describe_class`		
Framework adapters	`gemstone_py.web_core`, `gemstone_py.frameworks`, `gemstone_py.aio.fastapi`, `gemstone_py.aio.litestar`	`examples/fastapi/`, `examples/litestar/`, `examples/django/`	Framework Adapters
Examples	`gemstone_py.cli`	`examples/quickstart.py`, `examples/webstack/`, `examples/cookbook/`	Examples Guide, examples/README.md
Performance docs	`gemstone_py.benchmarks`, `gemstone_py.benchmark_compare`	`examples/native_backend/`	Performance
Native wheels	`gemstone_py.native`, `gemstone-py-native/`	`examples/native_backend/`	README, Release Checklist
Bootstrap	`gemstone_py.bootstrap`, `gemstone_py/_gemstone_side/`	`examples/quickstart.py`, `examples/cookbook/`	Setup Guide

Verification Gates

Before treating the plan3 batch as release-ready, run the local checks:

```
.venv/bin/python -m ruff check gemstone_py tests
.venv/bin/python -m mypy
.venv/bin/python -m pytest -q
./scripts/check_codegen.sh
```

Then run the live GemStone checks on a configured stone:

```
GS_RUN_LIVE=1 ./scripts/run_live_checks.sh
```

For release packaging, validate the native package workflow and the workbench extension release checks:

```
./scripts/run_native_checks.sh
cd vscode-gemstone-py-workbench
npm run release:preflight
npm run release:verify-vsix
```

Type-Safe Smalltalk Codegen

`gemstone-codegen` turns registered Python `Protocol` classes into concrete `TypedOop` wrappers. The generated files are ordinary Python modules, so IDEs can autocomplete methods and your application code no longer has to spell the same Smalltalk strings at every call site.

Install

The generator ships with `gemstone-py`:

```
python -m pip install gemstone-py
gemstone-codegen --help
```

For source checkouts:

```
python -m pip install -e ".[examples,dev]"
python -m gemstone_py.codegen --help
```

Define Protocols

Create a module with one or more `@gemstone_class(...)` protocols:

```
from typing import Protocol
from gemstone_py import gemstone_class, gemstone_selector

@gemstone_class("OkzBooking", async_=True)
class OkzBookingProto(Protocol):
    status: str

    @classmethod
    @gemstone_selector("findById:")
    def find_by_id(cls, booking_id: str) -> "OkzBookingProto":
        ...
```

```
    def mark_paid(self, at_posix_seconds: int) -> None:
        ...

    def yourself(self) -> "OkzBookingProto":
        ...

    def customer(self) -> "OkzCustomerProto":
        ...

    @gemstone_selector("transferTo:byUserId:")
    def transfer(self, user_id: int, by_user_id: int) -> None:
        ...

@gemstone_class("OkzCustomer", async_=True)
class OkzCustomerProto(Protocol):
    name: str
```

`async_=True` asks the generator to emit both sync and async wrappers.

Generate Wrappers

Run the generator from the repository or application root:

```
gemstone-codegen \
  --module examples.typed_access.codegen_demo.models \
  --output examples/typed_access/codegen_demo/generated \
  --clean
```

The output is meant to be checked in. That keeps reviews, editor indexing, and type checking predictable:

```
git add examples/typed_access/codegen_demo/generated
```

The generated package includes `.pyi` stubs and `py.typed` so type checkers treat the checked-in wrapper package as typed. `--clean` removes stale generated wrapper modules and stale wrapper stubs when a Protocol is renamed or deleted.

The generator keeps runtime code lightweight. It emits ordinary Python wrapper classes that call the existing session API; it does not add a registry, identity map, object cache, or global mapper.

Each generated wrapper class also exposes lightweight runtime metadata:

```
print(OkzBooking.__gemstone_protocol__)
print(OkzBooking.__gemstone_selectors__["find_by_id"])
```

Use that metadata for diagnostics, generated-wrapper audits, and UI tooling without parsing generated source files.

Use package generation for protocols that return other generated protocols. Single-class `generate_wrapper(...)` only has enough context to resolve self-typed returns.

Use `--check` in CI or pre-commit hooks to catch drift:

```
gemstone-codegen \
  --module examples.typed_access.codegen_demo.models \
  --output examples/typed_access/codegen_demo/generated \
  --check
```

The repository CI runs the same check through:

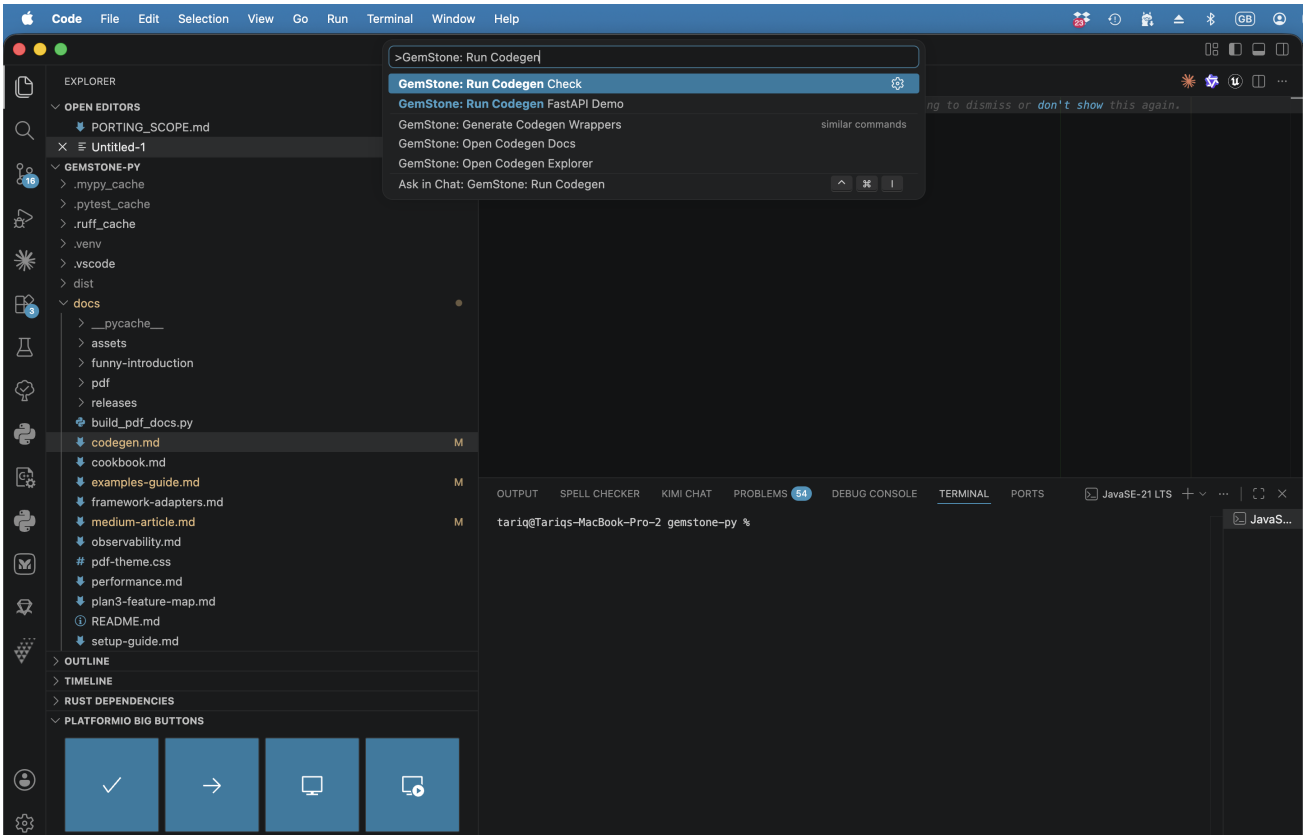
```
./scripts/check_codegen.sh
```

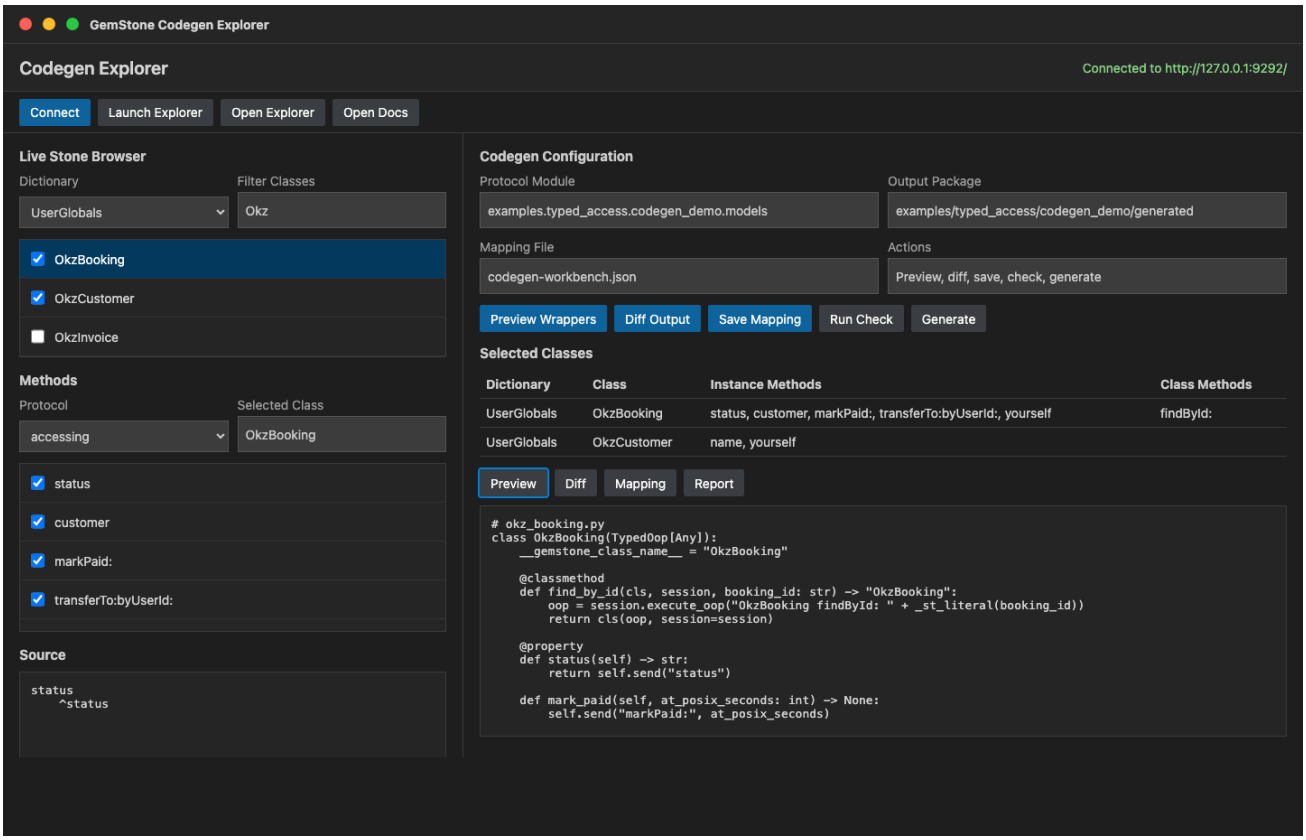
If your application uses `pre-commit`, it can consume the packaged hook:

```
repos:
  - repo: https://github.com/unicompute/gemstone-py
    rev: main
    hooks:
      - id: gemstone-codegen-check
```

Visual Codegen Explorer

The VS Code workbench includes `GemStone: Open Codegen Explorer` for the visual workflow around the same generator. It sits between the live `python-gemstone-database-explorer` class browser and the checked-in Protocol module. The browser explorer provides the stable `/codegen/*` JSON API; the VS Code workbench consumes that API instead of owning GemStone discovery or code-generation rules.





1. Start the database explorer with `gemstone-py: Launch Database Explorer`.
2. Run `GemStone: Open Codegen Explorer`.
3. Click `Connect` to load dictionaries from the live stone.

4. Browse dictionaries, classes, protocols/categories, methods, and source

   through `/codegen/dictionaries`, `/codegen/classes`, `/codegen/protocols`, `/codegen/methods`, and `/codegen/source`.

5. Select classes and instance/class-side methods as wrapper targets.
6. Save or load normalized selection JSON with fields, selectors, generated

   Python names, argument names, and return annotations.

7. Use `Preview Wrappers` to call `/codegen/preview` before writing files.
8. Use `Diff Output` to compare the preview output against the configured

   generated package.

9. Run `Run Check`, `Generate`, or `Run Demo` when the preview looks right.

The visual selection file is intentionally separate from the Protocol module. It records the live classes, fields, methods, class-side selectors, and edited Python metadata you chose while you design or review wrappers; the generated Python still comes from explicit `@gemstone_class(...)` Protocol definitions so the API remains reviewable and type-checkable. The database explorer normalizes this file with `/codegen/export-selection`, so the browser UI, VS Code workbench, and future tooling can share the same handoff format.

The demo mapping file is:

```
examples/typed_access/codegen_demo/codegen-workbench.example.json
```

It records this useful first selection in the current normalized schema:

Class	Fields	Class-side selectors	Instance selectors
`OkzBooking`	`status`	`findById:`	`customer`, `markPaid:`, `transferTo:byUserId:`, `yourself`
`OkzCustomer`	`name`	none	`yourself`

Use that mapping as the checklist when you browse the live classes, or import it into the explorer to restore the richer method metadata. Keep the Protocol module as the source of truth for the generated Python API.

Concrete Demo Workflow

The repository includes a complete, reviewable Codegen demo under `examples/typed_access/codegen_demo/`.

Preview generation without touching checked-in files:

```
python -m examples.typed_access.codegen_demo.preview
python -m examples.typed_access.codegen_demo.preview --show-source
```

Regenerate the checked-in package:

```
gemstone-codegen \
  --module examples.typed_access.codegen_demo.models \
  --output examples/typed_access/codegen_demo/generated \
  --clean
```

Verify that generated files are current:

```
gemstone-codegen \
  --module examples.typed_access.codegen_demo.models \
  --output examples/typed_access/codegen_demo/generated \
  --check
```

Run the generated-wrapper FastAPI demo:

```
python -m examples.typed_access.codegen_demo.run --reload
```

Probe the generated sync wrapper against a live stone:

```
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
python -m examples.typed_access.codegen_demo.live_probe --booking-id B-1001
```

Selector Mapping

Default mapping is intentionally small:

Python shape	Generated Smalltalk selector
`status`	`status`
`find_by_id(id)`	`findById:`
`mark_paid(at)`	`markPaid:`

Python shape	Generated Smalltalk selector
`transfer_to(user_id, by_user_id)`	`transferTo:byUserId:`

For selectors that do not map cleanly, use `@gemstone_selector(...)`. The explicit selector must have the same keyword count as the Python method has arguments.

Return Mapping

The generator uses simple Protocol annotations to choose the send path and the generated Python return type:

Protocol return annotation	Generated behaviour
no annotation	`perform_value(...)` / `session.eval(...)`, returns `Any`
`None`	sends the message and returns `None`
same Protocol class, wrapper class, or `Self`	uses `perform_oop(...)` or `execute_oop(...)` and wraps the returned OOP
another generated Protocol in the same module	uses `perform_oop(...)` or `execute_oop(...)` and lazily imports the target wrapper
simple builtins such as `str`, `int`, `float`, `bool`	value send with that return annotation

That means this method returns another typed wrapper:

```
def yourself(self) -> "OkzBookingProto":
    ...
```

and this method returns a generated `OkzCustomer` or `AsyncOkzCustomer` wrapper:

```
def customer(self) -> "OkzCustomerProto":
    ...
```

while this method is treated as a mutating command:

```
def mark_paid(self, at_posix_seconds: int) -> None:
    ...
```

Argument Conversion

Generated class-side wrappers turn simple Python literals into Smalltalk source only for the small supported set: strings, integers, floats, booleans, and `None`. Instance-side generated wrappers send raw OOP arguments through `perform_*`, so pass an existing raw OOP, `Oop` marker, or wrapper-managed OOP when the selector expects an object reference.

Keep complex query expressions, dynamic Smalltalk, and application-specific value conversion outside generated source. For scalar-ish Python values such as `datetime`, `date`, `Decimal`, or `UUID`, use the explicit converter registry described in the User Manual before passing the resulting `Oop` marker to the wrapper or session call.

Sync Usage

Generated class-side methods take a session and build the Smalltalk source:

```
from gemstone_py import GemStoneConfig, GemStoneSession
from examples.typed_access.codegen_demo.generated import OkzBooking

with GemStoneSession(config=GemStoneConfig.from_env()) as session:
    booking = OkzBooking.find_by_id(session, "B-1001")
    print(booking.status)
    booking.mark_paid(1_779_912_000)
    same_booking = booking.yourself()
    customer = booking.customer()
    print(customer.name)
```

That class-side call evaluates:

```
OkzBooking findById: 'B-1001'
```

Instance methods call `perform_value(...)` for value returns and `perform_oop(...)` for self-typed wrapper returns through the session associated with the returned `TypedOop`.

Async Usage

When the Protocol is registered with `async_=True`, the generator also emits an `Async...` wrapper:

```
from fastapi import Depends, FastAPI
from gemstone_py import GemStoneConfig
from gemstone_py.aio import AsyncSession
from gemstone_py.aio.fastapi import session_dependency
from examples.typed_access.codegen_demo.generated import AsyncOkzBooking
```

```
app = FastAPI()
get_gemstone =
session_dependency(config=GemStoneConfig.from_env(require_credentials=False))

@app.get("/bookings/{booking_id}")
async def booking_status(
    booking_id: str,
    session: AsyncSession = Depends(get_gemstone),
) -> dict[str, str]:
    booking = await AsyncOkzBooking.find_by_id(session, booking_id)
    return {"status": str(await booking.status())}
```

The repository includes a runnable version of that handler:

```
python -m examples.typed_access.codegen_demo.run --reload
```

With the server running, open:

```
http://127.0.0.1:8000/
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/bookings/B-1001
```

The root and docs routes are local FastAPI checks. The booking route requires GemStone credentials and a reachable stone because it evaluates `OkzBooking findById:`.

Current Scope

The first generator pass handles:

- annotated fields and `@property` methods as no-argument sends
- instance methods as `perform_value(...)` or `perform_oop(...)` sends based on

  return annotations

- class methods as class-side Smalltalk source strings
- self-returning class and instance methods as wrapped `TypedOop` results
- same-module cross-Protocol returns as lazily imported generated wrappers
- string, integer, float, boolean, and `None` literals in class-side calls
- conservative argument conversion that rejects unsupported source literals

  instead of guessing

- explicit selector overrides with `@gemstone_selector(...)`
- checked-in sync and async generated modules with `.pyi` stubs, `py.typed`,

  stale-file cleanup, and CI/pre-commit drift checks

- runtime source-Protocol and selector-map metadata on every generated wrapper
- a runnable FastAPI demo route backed by the generated async wrapper
- an opt-in live smoke test for generated wrappers against GemStone `Date`

It does not marshal arbitrary Python objects into class-side source literals or replace hand-written Smalltalk for complex queries. Use it for method-shaped object access and keep raw `session.eval(...)` or `SmalltalkBridge` calls where a full Smalltalk expression is clearer.

Adding a Framework Adapter

Use `gemstone_py.web_core` for request lifetime and transaction decisions. Adapters should stay thin: translate the framework's request hooks into `RequestScope` or `AsyncRequestScope`, then expose one helper that returns the current request session.

Keep retry policy in the application service layer. The adapter should own session acquisition and final commit/abort behaviour; it should not silently replay request handlers.

Sync Framework Shape

```
from gemstone_py import GemStoneConfig, RequestScope, TransactionPolicy
from gemstone_py.session_providers import GemStoneSessionPool

pool = GemStoneSessionPool(maxsize=4, config=GemStoneConfig.from_env())

def begin_request(request):
    request.gemstone_scope = RequestScope(
        session_provider=pool,
        transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
    )

def current_session(request):
    return request.gemstone_scope.session()

def end_request(request, exc=None, response_status=None):
    request.gemstone_scope.finalize(exc, response_status=response_status)
```

The scope commits on successful responses, aborts on exceptions or server-error status codes, and releases pooled sessions with the correct `clean` / `discard` flags.

When a specific write workflow needs bounded conflict replay, wrap that workflow with `retrying_transaction(...)` outside the request scope or in a service function that can safely rerun the whole unit of work:

```
from gemstone_py import retrying_transaction

def save_booking(session):
    ...

retrying_transaction(save_booking, config=GemStoneConfig.from_env(), attempts=5)
```

Do not retry an already-running request scope unless the whole request body can be safely replayed after aborting and reloading state.

Async Framework Shape

```
from gemstone_py import AsyncRequestScope, GemStoneConfig, TransactionPolicy
from gemstone_py.aio.pool import AsyncSessionPool

pool = AsyncSessionPool(maxsize=8, config=GemStoneConfig.from_env())

async def begin_request(scope):
    scope["gemstone_scope"] = AsyncRequestScope(
        session_provider=pool,
        transaction_policy=TransactionPolicy.COMMIT_ON_SUCCESS,
    )
    scope["gemstone_session"] = await scope["gemstone_scope"].session()

async def end_request(scope, exc=None, response_status=None):
    await scope["gemstone_scope"].finalize(exc, response_status=response_status)
```

Existing Adapters

Framework	Module	Pattern
Flask	`gemstone_py.frameworks.flask`	request-local sync session
Django	`gemstone_py.frameworks.django`	middleware-managed sync session
FastAPI	`gemstone_py.aio.fastapi`	async dependency and ASGI middleware
Litestar	`gemstone_py.aio.litestar`	async dependency and ASGI middleware

Keep optional framework imports inside app/example code or function bodies, not at package import time. The adapter module should remain importable in a plain `gemstone-py` installation.

One GemStone session belongs to one active execution path. Framework adapters should make session sharing explicit through a pool or request-local provider, not by storing a session in a global variable.

Observability

`gemstone-py` can emit tracing spans, metrics, and slow-operation log records from the core session API. The default is no-op, so existing applications do not need to configure anything and do not pull in observability dependencies.

Install Optional Adapters

The built-in OpenTelemetry and Prometheus adapters use optional dependencies:

```
python -m pip install "gemstone-py[observability]"
```

You can also pass your own objects that implement the small protocols in `gemstone_py.observability`.

Trace GCI Calls

Pass a tracer when creating a session:

```
from opentelemetry import trace
from gemstone_py import GemStoneConfig, GemStoneSession, OpenTelemetryTracer

tracer = OpenTelemetryTracer(trace.get_tracer("my-app.gemstone"))

with GemStoneSession(config=GemStoneConfig.from_env(), tracer=tracer) as session:
    session.execute("1 + 1")
    session.perform_oop(session.resolve("System"), "myUserProfile")
```

Each observed operation creates a span named like:

```
gemstone.session.execute_oop
gemstone.session.perform_oop
gemstone.session.bulk_perform_oop
gemstone.session.bulk_perform_calls_oop
gemstone.session.commit
gemstone.session.abort
```

Common span attributes include:

Attribute	Meaning
`operation`	Session operation name, such as `execute_oop` or `perform_oop`.
`stone`	Configured GemStone stone name.
`host`	Configured GemStone host.
`status`	`ok` or `error`.
`duration_ms`	Wall-clock duration for the operation.
`selector`	Smalltalk selector for `perform_*` calls.
`argc`	Argument count for `perform_*` calls.
`receiver_count`	Receiver count for `bulk_perform_*` calls.
`call_count`	Call count for `bulk_perform_calls_*` calls.
`source_length`	Source string length for eval/execute calls.

The instrumentation deliberately records source length rather than full Smalltalk source, so normal traces do not expose application data or secrets.

Conflict And Retry Reports

Transaction retry helpers do not emit logs by themselves. Pass an `on_conflict=` listener so application code decides where retry diagnostics go:

```
import logging
from gemstone_py import retrying_transaction

logger = logging.getLogger("my-app.gemstone")

def log_conflict(retry):
    logger.warning("gemstone commit conflict\n%s",
retry.format(include_summaries=False))

retrying_transaction(work, config=config, attempts=5, on_conflict=log_conflict)
```

Use `retry.to_dict(...)` when a structured logger should receive JSON-friendly fields. Use `include_summaries=False` if object summaries may contain application data that should not be copied into logs.

Record Metrics

Pass a metrics collector alongside the tracer, or use metrics on their own:

```
from gemstone_py import GemStoneConfig, GemStoneSession, PrometheusMetrics

metrics = PrometheusMetrics()

with GemStoneSession(config=GemStoneConfig.from_env(), metrics=metrics) as session:
    session.execute("1 + 1")
```

The session emits:

Metric	Labels	Meaning
`gemstone_py_session_operations`	`operation`, `status`	Operation counter.
`gemstone_py_session_duration_ms`	`operation`, `status`	Duration samples in milliseconds.

For existing Prometheus setup, pass your service's registry to `PrometheusMetrics(registry=...)`.

Pool And Query Signals

`GemStoneSessionPool`, `GemStoneThreadLocalSessionProvider`, and `AsyncSessionPool` accept the same `metrics=` and `tracer=` objects:

```
from gemstone_py import GemStoneConfig, GemStoneSessionPool, PrometheusMetrics

metrics = PrometheusMetrics()
pool = GemStoneSessionPool(
    maxsize=4,
    config=GemStoneConfig.from_env(),
    metrics=metrics,
)
```

The pool emits:

Metric	Labels	Meaning
`gemstone_py_pool_events`	`event`, `provider`, `provider_type`, optional `reason`	Pool lifecycle/event counter.

Metric	Labels	Meaning
`gemstone_py_pool_acquire_wait_ms`	`provider`, `provider_type`	Time spent waiting for a pooled session.

Pool spans are named like:

```
gemstone.pool.session_acquired
gemstone.pool.session_released
gemstone.pool.session_discarded
gemstone.pool.acquire_timeout
```

Chunked query iteration emits session-level spans when tracing is configured on the session:

```
gemstone.session.query_iter_chunk
gemstone.session.query_iter
```

Those spans include the collection name, chunk size, chunk range, number of chunks fetched, and total yielded rows.

Slow Operation Log

Enable structured warning logs for slow operations:

```
import logging
from gemstone_py import GemStoneConfig, GemStoneSession

logging.basicConfig(level=logging.WARNING)

with GemStoneSession(
    config=GemStoneConfig.from_env(),
    slow_query_threshold_ms=100.0,
) as session:
    session.execute("1 + 1")
```

Slow records are written through:

```
gemstone_py.slow_queries
```

The log record includes the operation, duration, stone, host, session id, status, and error type when one is available.

Async Sessions

`AsyncSession` creates and uses the same underlying `GemStoneSession`, so pass the same keyword arguments:

```
from opentelemetry import trace
from gemstone_py import GemStoneConfig, OpenTelemetryTracer
from gemstone_py.aio import AsyncSession

tracer = OpenTelemetryTracer(trace.get_tracer("my-app.gemstone"))

async with AsyncSession.connect(
    config=GemStoneConfig.from_env(),
    tracer=tracer,
    slow_query_threshold_ms=100.0,
) as session:
    await session.execute("1 + 1")
```

Custom Collectors

The protocols are intentionally small:

```
class MyMetrics:
    def increment(self, name, labels=None, value=1):
        ...

    def record_duration(self, name, labels, duration_ms):
        ...
```

This keeps the core package independent from any one telemetry stack while still making the production hooks explicit.

Performance

`gemstone-py` keeps performance claims tied to benchmark artifacts. The repository ships a maintained benchmark CLI, committed baselines, and a comparison workflow so performance regressions can be reviewed as data instead of anecdotes.

Current Committed Baseline

The current public baseline is `.github/benchmarks/baseline.json`. It was generated on 2026-04-20 with:

Field	Value
Platform	`macOS-26.3.1-arm64-arm-64bit-Mach-O`
Python	`CPython 3.14.3`
Stone	`gs64stone` on `localhost`
Entries	`200`
Search runs	`10`
Suites	`persistent_root`, `gscollection`, `gstore`, `rchash`

These numbers are a small smoke-sized profile, not a universal capacity claim. They are useful as a reference point and as a regression baseline for the configured self-hosted GemStone runner.

Suite	Operation	Count	Elapsed	Approx. per operation	Ops/s	Not
`persistent_root`	`write_mapping_commit`	200	0.1099s	0.550ms	1,819	Wri Pyth map com
`persistent_root`	`mapping_keys`	200	0.0006s	0.003ms	337,316	Rea com map
`gscollection`	`bulk_insert_and_index_commit`	200	0.0980s	0.490ms	2,040	Bull reco crea inde
`gscollection`	`indexed_search`	10	0.0126s	1.263ms	792	`mat`
`gstore`	`batch_write`	200	0.0929s	0.464ms	2,153	Tra key writ
`gstore`	`snapshot_read`	200	0.0689s	0.344ms	2,903	Tra key
`rchash`	`populate_commit`	200	0.0159s	0.079ms	12,581	Pop `RCH` com

Suite	Operation	Count	Elapsed	Approx. per operation	Ops/s	Not
`rchash`	`items`	200	0.0013s	0.006ms	154,644	Rea com hash

Run The Benchmarks

Run the default live benchmark suite against your configured stone:

```
./scripts/run_benchmarks.sh
gemstone-benchmarks --entries 500 --search-runs 20
```

Capture a JSON artifact:

```
./scripts/run_benchmarks.sh --json --output benchmark-report.json
```

The `gscollection` suite records both the legacy materialized path and the streaming path:

- `indexed_search` and `indexed_search_iter` compare list-returning indexed search with chunked indexed iteration.
- `all_materialize` records `collection.all()` latency and peak Python allocation.
- `iter_stream_count` records the same full-collection pass while streaming through `collection.iter()`.

Use a larger entry count, such as `--entries 50000`, when you want to compare large-result memory behavior rather than smoke-test latency.

Round-Trip Reduction APIs

The lightweight performance direction is to reduce avoidable GCI round trips without adding a client-side identity map. The main helpers are:

- `PersistentRoot.get_many(...)` and `PersistentRoot.update_many(...)` for top-level `UserGlobals` batches.

- `GsDict.get_many(...)` and `GsDict.update_many(...)` for nested `StringKeyValueDictionary` batches.
- `GemStoneSession.bulk_perform_value(...)` and `bulk_perform_oop(...)` for one selector across many receivers.
- `GemStoneSession.bulk_perform_calls_value(...)` and `bulk_perform_calls_oop(...)` for mixed receiver/selector calls.
- `perform_many_value(...)`, `perform_many_oop(...)`, `perform_calls_value(...)`, and `perform_calls_oop(...)` as readability aliases.

Use these when the unit of work is already a batch. They are not a substitute for good repository-side indexing or a reason to pull large object graphs into Python.

Compare two saved reports:

```
gemstone-benchmark-compare baseline.json candidate.json
gemstone-benchmark-compare baseline.json candidate.json --json --output benchmark-compare.json
```

Select the committed baseline that matches the generated report metadata:

```
python -m gemstone_py.benchmark_baselines benchmark-report.json \
  --manifest .github/benchmarks/index.json
```

Register a new accepted baseline:

```
gemstone-benchmark-baseline-register benchmark-report.json
```

Native Backend Microbenchmark

The `gci` suite compares low-level helper-call overhead for the pure `ctypes` backend and the optional PyO3 native backend. It does not require a live stone:

```
gemstone-benchmarks --suite gci --entries 1000000
```

To compare real GemStone workloads through each backend, run the same live benchmark twice with an explicit backend and compare the artifacts:

```
GEMSTONE_PY_GCI_BACKEND=ctypes gemstone-benchmarks --json --output ctypes-report.json
GEMSTONE_PY_GCI_BACKEND=native gemstone-benchmarks --json --output native-report.json
gemstone-benchmark-compare ctypes-report.json native-report.json
```

CI Policy

The manual `Benchmarks` workflow runs on the GemStone-capable self-hosted runner, uploads the fresh benchmark report, selects the best committed baseline, and enforces configured regression thresholds with `gemstone-benchmark-compare`.

The committed thresholds are deliberately per-operation rather than one global number. Write-heavy operations and indexed search have different run-to-run jitter, so a single global threshold either hides meaningful regressions or fails on normal noise.

When updating these numbers, commit both the new baseline report and the manifest update under `.github/benchmarks/`.

> This PDF was generated locally from the Markdown sources in `docs/`. If you edit the source files, rerun `python docs/build_pdf_docs.py`.